

OpenStitch Studio

Numérisation open source pour broderie machine

Documentation utilisateur et technique

Version du logiciel : 0.1.0 · Version de la documentation : 1.0

Générée le 2026-07-26

Licence : Apache-2.0

Nom temporaire du projet (remplaçable, ADR-001)

Informations éditoriales

Élément	Valeur
Statut du document	Version initiale, générée automatiquement à partir des sources
Version de la documentation	1.0
Version du logiciel documentée	0.1.0 (constante <code>kAppVersion</code> , <code>libs/core/include/openstitch/core/app_info.hpp</code>)
Date de génération	2026-07-26
Licence du logiciel	Apache-2.0 (LICENSE)
Licence de la documentation	Apache-2.0 (même dépôt)
Dépôt	Dépôt Git local ; aucun remote public déclaré à ce jour

Historique des révisions

Version	Date	Changements
1.0	2026-07-26	Première édition complète : audit du dépôt, guides utilisateur et technique, référence des modules, format DST, format projet.

Signaler une erreur

La documentation est générée depuis les fichiers Markdown de `docs/source/`. Pour corriger une erreur, modifiez le fichier source concerné puis régénérez le PDF (voir *Système de génération de la documentation* dans `docs/README.md`). Les affirmations techniques sont tracées vers le code réel dans chaque section « Implémentation associée ».

Table des matières

Les titres sont cliquables. Les lecteurs PDF affichent aussi les signets (panneau latéral) pour une navigation détaillée par chapitre et section.

[À propos de ce document](#)

[Introduction et présentation générale](#)

[Guide de prise en main](#)

[Installation et premier démarrage](#)

[Guide utilisateur détaillé](#)

[Traitement d'image](#)

[Segmentation](#)

[Vectorisation](#)

[Objets de broderie](#)

[Génération de points — point droit et fondations](#)

[Colonne satin](#)

[Remplissage tatami](#)

[Retouche des points et de la géométrie](#)

[Palettes et fils](#)

[Simulation de couture](#)

[Analyse et validation](#)

[Format DST \(Tajima\)](#)

[Format de projet `.osp`](#)

[Architecture logicielle](#)

[Référence des modules](#)

[Modèle de données](#)

[Algorithmes](#)

[Compilation et développement](#)

[Guide du développeur](#)

[Tests](#)

[Guide de contribution](#)

[Dépannage](#)

[Limitations et état des fonctionnalités](#)

[Roadmap](#)

[Glossaire](#)

[Licences](#)

[Annexe — Audit du dépôt](#)

À propos de ce document

Cette documentation décrit **OpenStitch Studio**, une application de bureau libre de numérisation pour broderie machine. Elle couvre l'usage du logiciel, les concepts de broderie qu'il met en œuvre, son architecture logicielle, ses formats de fichiers et sa compilation.

Le document s'adresse à quatre publics : l'**utilisateur débutant**, l'**utilisateur avancé en broderie**, le **développeur contributeur** et le **mainteneur**. Chaque chapitre indique clairement à qui il s'adresse en priorité.

Principe de véracité

Toutes les affirmations sur le fonctionnement du logiciel sont vérifiées dans le code réel du dépôt. Lorsqu'une capacité était prévue mais n'est pas encore disponible, elle est marquée explicitement :

- **Implémenté** : disponible et testé ;
- **Partiellement implémenté** : présent mais incomplet ;
- **Expérimental** : présent, non stabilisé ;
- **Prévu** : conçu dans l'architecture, non implémenté ;
- **Non implémenté** : absent.

L'annexe *Audit du dépôt* recense l'inventaire factuel qui sert de base à ce document. Chaque chapitre technique se termine par une section **Implémentation associée** listant les fichiers, classes, fonctions et tests réels.

Comment lire ce document

- Vous voulez **utiliser** le logiciel : lisez *Introduction*, *Installation*, *Guide de prise en main*, puis *Guide utilisateur détaillé*.
- Vous voulez **comprendre la broderie** telle qu'implémentée : lisez *Concepts* (points, satin, tatami) et *Pipeline image vers broderie*.
- Vous voulez **contribuer** : lisez *Architecture*, *Référence des modules*, *Modèle de données*, *Algorithmes*, *Compilation et développement*, *Tests*.
- Vous voulez **maintenir** : ajoutez *Limitations et roadmap*, *Licences* et le *Guide de contribution*.

Note : Ce document est généré automatiquement à partir des fichiers Markdown de `docs/source/`. Ne le modifiez pas directement dans le PDF.

Introduction et présentation générale

Vision du projet

OpenStitch Studio vise à offrir, **gratuitement et en logiciel libre**, la chaîne de travail qui va d'une **image matricielle** jusqu'à un **fichier de broderie machine** au format DST (Tajima). Le projet est écrit principalement en C++ moderne, fonctionne localement (sans compte, sans cloud, sans télémétrie) et sépare strictement son cœur métier de son interface graphique.

Le nom « OpenStitch Studio » est temporaire et remplaçable (décision ADR-001) : il n'apparaît en dur qu'à un seul endroit du code (kAppName).

Cas d'usage

- Transformer un logo ou une illustration simple en motif brodable.
- Segmenter une image en zones de couleur et les convertir en objets de broderie éditables (contours, remplissages, colonnes satin).
- Générer, prévisualiser et analyser les points avant de produire un fichier DST.
- Relire un fichier DST existant et l'inspecter.

La chaîne de travail en un coup d'œil

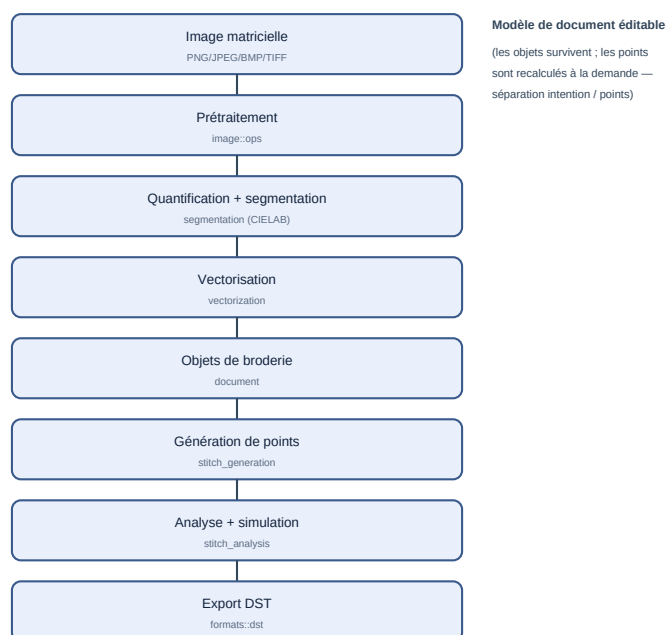


Figure 1 — Le pipeline image → broderie et les modules responsables.

L'idée centrale est la **séparation entre l'intention et le résultat** : la géométrie éditable (objets vectoriels et de broderie) est conservée dans un modèle de document ; les **points** sont recalculés à la

demande à partir de cette géométrie. Modifier un paramètre régénère les points sans détruire l'objet source.

Quatre représentations distinctes

Il est essentiel de distinguer quatre choses que le logiciel manipule :

Représentation	Ce que c'est	Type/Module
Image	pixels RGBA	<code>image::Image</code>
Région	zone de couleur connexe issue de la segmentation	<code>segmentation::Region</code>
Objet vectoriel	contours (extérieur + trous), éditables	<code>document::VectorObject</code>
Objet de broderie	intention de couture (type de point + paramètres)	<code>document::EmbroideryObject</code>
Séquence de points	commandes machine (Stitch/Jump/...)	<code>stitch::StitchSequence</code>

Un DST, lui, ne contient **que** la séquence de points : il ne conserve **pas** les objets éditables (voir le chapitre *Format DST*).

Formats

- **Entrée image** : PNG, JPEG, BMP, TIFF.
- **Broderie** : DST en import et export.
- **Diagnostic** : SVG en export.
- **Projet** : `.osp` (archive contenant tout le document éditable).

Limites générales

Le logiciel est un socle complet et testé, mais **il n'a pas été validé sur une machine à broder réelle**. Les heuristiques physiques (compensation de tirage, densité) sont paramétrables mais approximatives, et certaines conventions DST varient selon les machines. Voir le chapitre *Limitations et roadmap*.

Limitation : Certaines fonctionnalités décrites dans l'étude de cadrage initiale (palette de fils réelle, remplissages courbes, profils machine avancés, édition manuelle des points) sont **prévues mais non implémentées**. Elles sont signalées comme telles tout au long du document.

Philosophie open source

Le projet est sous licence **Apache-2.0** (permissive avec clause de brevets). Ce choix a une conséquence importante : le code sous licence copyleft (par exemple Ink/Stitch, en GPL) **ne peut pas** être copié dans ce dépôt ; seuls les algorithmes publiés et les bibliothèques permissives sont réutilisables. Voir *Licences*.

Implémentation associée

- `libs/core/include/openstitch/core/app_info.hpp` — nom et version.
- `docs/phase0/` — étude de cadrage et 14 décisions d'architecture (ADR).
- `README.md` — résumé du projet (établi à un jalon antérieur).

Guide de prise en main

Public : utilisateur débutant. Ce tutoriel suit un flux réaliste, **adapté à ce qui est réellement implémenté**. Les étapes non disponibles dans l'interface sont signalées.

Vue d'ensemble

Le flux typique est : importer une image → la préparer → la segmenter en régions → vectoriser → créer des objets de broderie (ou laisser l'auto-numérisation le faire) → régler les points → organiser l'ordre → analyser → exporter en DST.

1. Ouvrir une image

Fichier → **Ouvrir une image...** (Ctrl+O). Choisissez un PNG, JPEG, BMP ou TIFF. Un logo simple à quelques couleurs franches donne les meilleurs résultats.

2. Choisir les dimensions physiques

Un dialogue d'import demande la **taille physique** en millimètres (largeur et hauteur), avec l'option « conserver les proportions ». La valeur par défaut suppose 96 dpi. C'est ici que se fixe la résolution de travail (mm par pixel) — elle ne changera plus ensuite.

Conseil : visez une taille qui tient dans le cadre affiché (100 × 100 mm par défaut) ; sinon le motif dépassera du tambour.

3. Préparer l'image

Menu **Image** : niveaux de gris, luminosité/contraste (avec aperçu), débruitage, symétries, rotations 90°, recadrage (par sélection au rectangle) et quantification des couleurs. Toutes ces opérations sont **non destructives** : l'original est conservé, chaque opération s'annule (Ctrl+Z).

4. Réduire les couleurs et segmenter

Segmentation → **Segmenter l'image...** : choisissez le nombre maximal de couleurs et la taille minimale de région. Le logiciel quantifie en espace perceptuel CIELAB puis extrait les **régions connexes**. Activez **Afficher la carte des régions** pour les visualiser.

5. Nettoyer les régions

Cliquez une région pour la sélectionner (ses infos s'affichent dans la barre d'état). Vous pouvez la **supprimer** (elle redevient du fond), la **recolorer**, ou en **fusionner** deux (« Fusionner avec... » puis clic sur la cible).

6. Vectoriser

Avec une région sélectionnée : **Segmentation** → **Convertir la région en objet vectoriel**. Le contour (et ses trous) devient un objet éditable ; ses **nœuds** apparaissent et se déplacent à la souris.

7. Créer des objets de broderie

Sur un objet vectoriel sélectionné, menu **Broderie** :

- **Créer un objet de point de contour...** (point simple, double ou triple) ;
- **Créer un remplissage tatami...** (densité, longueur, angle) ;
- **Créer une colonne satin...** (densité, compensation, sous-couche) — avec un avertissement si la colonne est trop large.

Alternative rapide : **Segmentation** → **Numériser automatiquement** crée des objets pour toutes les régions selon leur forme (satin pour les bandes fines, tatami pour les zones larges, contour pour les petites).

8. Régler les points et les couleurs

Les points se régénèrent automatiquement à chaque changement. La couleur d'un objet reprend celle de sa région.

Limitation : il n'existe pas encore de **palette de fils** réelle ni de sélection de fil par référence fabricant (voir *Palettes et fils*).

9. Organiser l'ordre de couture

Le panneau **Ordre de couture** liste les objets ; vous pouvez les monter/descendre, les verrouiller, et appliquer une stratégie automatique (par couleur, par proximité) avec une estimation de coût.

10. Simuler et analyser

La **barre de simulation** rejoue la couture point par point (lecture/pause, curseur). **Analyse** → **Analyser le motif** (F5) liste les problèmes détectés (points trop courts/longs, sauts trop longs, hors cadre) ; un double-clic centre la vue sur le problème.

11. Exporter en DST

Fichier → **Exporter en DST...** Le logiciel rappelle qu'un DST **ne conserve pas** les objets éditables : gardez aussi votre projet `.osp`.

12. Vérifier le fichier

En ligne de commande : `openstitch-cli stats motif.dst` affiche points, sauts, dimensions et longueur de fil estimée ; `openstitch-cli dst2svg motif.dst apercu.svg` produit un aperçu vectoriel.

Implémentation associée

- `apps/desktop/main_window.cpp` — tous les menus et actions ci-dessus.
- `apps/desktop/import_dialog.cpp` — dialogue de taille physique.
- `libs/autodigitize/src/autodigitize.cpp` — numérisation automatique.
- `apps/cli/main.cpp` — `stats`, `dst2svg`.

Installation et premier démarrage

Public : utilisateur débutant, développeur.

À ce jour, **aucun binaire pré-compilé n'est distribué** : l'installation se fait par **compilation depuis les sources**. Cette section décrit la mise en place minimale ; la compilation détaillée est traitée dans *Compilation et développement*.

Systèmes supportés

Système	Statut
Windows 10 / 11 (x64)	Cible principale (application complète)
Linux	Cœur + CLI uniquement (garde-fou de portabilité, vérifié en CI) — pas l'interface Qt à ce stade
macOS	Prévu, non vérifié

Prérequis (Windows)

Outil	Version	Rôle
Visual Studio 2022 ou plus récent (charge « Développement Desktop C++ »)	MSVC v143+	Compilateur
CMake	≥ 3.27	Configuration du build
Git	récent	Récupération du code et de vcpkg
vcpkg	bootstrappé	Dépendances (OpenCV, Clipper2, ...)
Qt	6.8 LTS, binaires <code>msvc2022_64</code>	Interface graphique
Espace disque	~10 Go	vcpkg compile OpenCV au premier configure

Mise en place

1. Installer vcpkg et définir `VCPKG_ROOT` :

```
git clone https://github.com/microsoft/vcpkg.git C:\dev\vcpkg
C:\dev\vcpkg\bootstrap-vcpkg.bat -disableMetrics
setx VCPKG_ROOT C:\dev\vcpkg
```

2. Installer Qt 6.8 (sans compte, via `aqtinstall`) et définir `QT_ROOT` :

```
python -m pip install --user aqtinstall
python -m aqt install-qt windows desktop 6.8.3 win64_msvc2022_64 -O C:\Qt -m qtimageformats
setx QT_ROOT C:\Qt\6.8.3\msvc2022_64
```

3. Configurer, compiler, tester (nouveau terminal pour prendre les variables) :

```
cmake --preset msvc
cmake --build --preset msvc-debug
ctest --preset msvc-debug
```

Les exécutables produits :

- `build\msvc\apps\desktop\Debug\openstitch.exe` — application graphique ;
- `build\msvc\apps\cli\Debug\openstitch-cli.exe` — outil en ligne de commande.

Démarrage

Lancez `openstitch.exe`. La fenêtre principale s'ouvre sur un **canevas** vide, avec des règles graduées en millimètres et un **cadre** de broderie de 100 × 100 mm par défaut. Utilisez **Fichier** → **Ouvrir une image...** pour commencer.

Emplacement des données, configuration, désinstallation

- **Données** : le logiciel ne stocke rien en dehors des fichiers projet `.osp` et des fichiers exportés que vous choisissez. *Information non déterminée dans le dépôt* : aucun répertoire de configuration utilisateur n'est créé (pas de persistance de préférences repérée dans le code).
- **Configuration** : les variables d'environnement `VCPKG_ROOT` et `QT_ROOT` concernent la **compilation**, pas l'exécution.
- **Désinstallation** : supprimez le dossier de build et, le cas échéant, `vcpkg` et Qt. Aucune entrée de registre n'est écrite par l'application.

Vérification de l'installation

- `ctest --preset msvc-debug` doit rapporter **100 % des tests réussis**.
- `openstitch-cli.exe --version` affiche la version.
- `openstitch-cli.exe info une-image.png` affiche les métadonnées d'une image.

Avertissement : au **premier** `cmake --preset msvc`, `vcpkg` compile OpenCV et d'autres dépendances, ce qui peut prendre 10 à 30 minutes. Les compilations suivantes réutilisent le cache binaire de `vcpkg`.

Implémentation associée

- `docs/build-windows.md` — guide de compilation d'origine.
- `CMakePresets.json` — presets `msvc` et `linux-core`.
- `vcpkg.json` — dépendances verrouillées par baseline.
- `apps/cli/main.cpp` — sous-commande `info`, drapeau `--version`.

Guide utilisateur détaillé

Public : utilisateur débutant et avancé. Ce chapitre documente chaque menu, outil et raccourci **réellement présents** dans `apps/desktop/main_window.cpp`.

Le canevas

La zone centrale est un canevas dont l'unité est le **millimètre** (origine au centre, Y vers le haut). Des **règles** graduées en mm bordent le haut et la gauche ; une **grille** adaptative et un **cadre** de broderie (rectangle rouge, 100 × 100 mm par défaut) sont dessinés. La position du curseur en mm apparaît dans la barre d'état.

- **Zoom** : molette (ancrée sous le curseur) ou menu Affichage.
- **Déplacement** : cliquer-glisser (main).
- **Recadrage/sélection** : bascule en mode rectangle quand l'outil de recadrage est actif.

Le rendu est organisé en deux couches (image/vecteurs/régions d'une part, points d'autre part) pour rester fluide sur de gros motifs.

Menu Fichier

Action	Raccourci	Effet
Ouvrir une image...	Ctrl+O	Charge PNG/JPEG/BMP/TIFF puis demande la taille physique
Enregistrer le projet...	Ctrl+S	Écrit un fichier <code>.osp</code> (tout le document)
Ouvrir un projet...	—	Recharge un <code>.osp</code>
Exporter en DST...	—	Écrit un fichier <code>.dst</code> (points uniquement)
Importer un DST...	—	Relit un <code>.dst</code> comme séquence de points
Quitter	Ctrl+Q	Ferme l'application

Menu Édition

Action	Raccourci	Effet
Annuler	Ctrl+Z	Défait la dernière opération (le libellé nomme l'action)
Rétablir	Ctrl+Y	Refait l'opération annulée

L'annulation couvre les opérations d'image, la segmentation (segmenter, fusionner, supprimer, recolorer), la création et le déplacement de nœuds vectoriels, la création d'objets de broderie et le réordonnement.

Menu Image

Toutes ces opérations sont non destructives (pile de transformations sur l'original) :

- Niveaux de gris ;
- Luminosité/contraste... (aperçu en direct) ;
- Débruitage léger / moyen (médian) ;
- Quantifier les couleurs... (k-means, nombre de couleurs 2–64) ;
- Symétrie horizontale / verticale ;
- Rotation 90° horaire / antihoraire ;
- Recadrer (sélection) — dessinez un rectangle sur le canevas.

Menu Segmentation

- Segmenter l'image... (nombre de couleurs, taille min de région) ;
- Afficher la carte des régions (bascule) ;
- Fusionner avec... (puis clic sur la région cible) ;
- Supprimer la région sélectionnée (Suppr) — la région redevient du fond ;
- Recolorer la région sélectionnée... ;
- Convertir la région en objet vectoriel.

Sélection : cliquez une région ; ses statistiques (pixels, mm², RGB) s'affichent.

Menu Broderie

- Numérisation automatique — crée des objets pour toutes les régions ;
- Créer un objet de point de contour... (longueur, type simple/double/triple) ;
- Créer un remplissage tatami... (espacement, longueur, angle) ;
- Créer une colonne satin... (densité, compensation, sous-couche centrale) ;
- Statistiques... (points, sauts, coupes, changements de couleur, dimensions, longueur de fil).

Avertissement : si une colonne satin dépasse la largeur recommandée, un dialogue propose de continuer ou de préférer un remplissage tatami.

Menu Affichage

Action	Raccourci	Effet
Afficher les points	—	Bascule l'affichage des points grisés
Afficher les vecteurs	—	Bascule l'affichage des étapes vectorielles
Zoom avant	Cm++	Agrandir
Zoom arrière	Cm--	Réduire
Ajuster les contrastes	Cm0	Contraste le plus fort le contraste

Les points cousus sont tracés en trait continu foncé ; les **sauts** en pointillés orange ; des pastilles marquent les pénétrations (masquées au-delà de 4000 points pour la fluidité, et pendant la simulation).

Menu Analyse et panneau

Analyser le motif (F5) remplit le panneau *Analyse* (dock) avec les problèmes détectés, triés par gravité. Un double-clic sur un problème centre la vue sur sa localisation.

Panneau Ordre de couture

Dock listant les objets de broderie dans l'ordre. Vous pouvez monter/descendre un objet, le verrouiller (il ne bougera plus lors de l'optimisation), et appliquer une stratégie (ordre du document, par couleur, par proximité, couleur puis proximité). Un libellé affiche le coût estimé.

Barre de simulation

Boutons de lecture/pause et un curseur qui révèle la couture jusqu'à un index de point, avec un repère d'aiguille. La barre n'apparaît que lorsqu'une séquence de points existe.

Erreurs et messages

Les opérations impossibles (recadrage hors image, satin non constructible, export d'une séquence vide...) affichent un message clair et **n'appliquent rien**. Aucune entrée utilisateur incorrecte ne fait planter l'application.

Note : Les raccourcis proviennent des séquences standard de Qt (`QKeySequence::Open`, `Save`, `Undo`, `Redo`, `Delete`, `ZoomIn`, `ZoomOut`, `Quit`) et de `Ctrl+0`, `F5` définis explicitement.

Implémentation associée

- `apps/desktop/main_window.cpp` — construction des menus, actions, docks, barre.
- `apps/desktop/canvas_view.cpp` — zoom, déplacement, grille, cadre, rendu points.
- `apps/desktop/ruler.cpp` — règles en mm.
- `apps/desktop/import_dialog.cpp`, `brightness_dialog.cpp` — dialogues.

Traitement d'image

Public : utilisateur avancé, développeur. État : **Implémenté** (non destructif).

Principe

L'image importée est convertie en une **image de travail RGBA 8 bits** (`image::Image`). L'original n'est **jamais** modifié : le document conserve l'original et une **pile d'opérations** (`std::vector<ImageOp>`) rejouée à la demande par `apply_pipeline`. C'est ce qui rend toutes les retouches annulables.

Chargement

`load_image` lit le fichier via OpenCV (`cv::imdecode` sur les octets, ce qui gère les chemins Windows non-ASCII), gère 8 et 16 bits, niveaux de gris + alpha, et normalise tout en RGBA. `read_image_info` renvoie largeur, hauteur, canaux, présence d'alpha et format.

Opérations disponibles

Chaque opération est une alternative du variant `image::ImageOp` :

Opération	Type	Paramètres	Effet
Recadrage	<code>CropOp</code>	x, y, largeur, hauteur (px)	Découpe (borné à l'image)
Symétrie	<code>FlipOp</code>	horizontal/vertical	Miroir
Rotation 90°	<code>Rotate90Op</code>	quarts de tour 1–3	Rotation
Niveaux de gris	<code>GrayscaleOp</code>	—	Gris (alpha conservé)
Luminosité/contraste	<code>BrightnessContrastOp</code>	–100..100	Ajuste (alpha conservé)
Débruitage	<code>MedianDenoiseOp</code>	force 1–2 (noyau 3/5)	Médian
Quantification	<code>QuantizeOp</code>	2–64 couleurs	k-means déterministe (RGB)

Note : Le **rééchantillonnage** (redimensionnement de la résolution de travail) est volontairement absent de cette pile : il changerait le rapport mm/pixel. La taille physique se fixe à l'import (voir *Modèle physique*).

Déterminisme

La quantification fixe la graine du générateur aléatoire d'OpenCV (`cv::setRNGSeed`), donc deux exécutions donnent le même résultat — prérequis des tests et de la reproductibilité.

Aperçu et annulation

La luminosité/contraste offre un **aperçu en direct** (le dialogue émet un signal, la fenêtre recalcule l'affichage sans modifier le document tant que l'utilisateur n'a pas validé). Toute opération validée passe par une commande d'annulation (`AppendImageOpCommand`).

Erreurs possibles

- Recadrage hors de l'image → message, aucune modification.
- Nombre de couleurs hors bornes → refus.
- Fichier illisible/corrompu → erreur `InvalidFile`, pas de crash.

Implémentation associée

- `libs/image/include/openstitch/image/ops.hpp` — le variant `ImageOp`.
- `libs/image/src/ops.cpp` — `apply_op`, `apply_pipeline` (OpenCV encapsulé).
- `libs/image/src/load.cpp` — `load_image`, `read_image_info`, `encode_png`, `decode_image`.
- `libs/commands/include/openstitch/commands/project_commands.hpp` — `AppendImageOpCommand`.
- Tests : `tests/unit/image/test_ops.cpp`, `test_image_load.cpp`.

Segmentation

Public : utilisateur avancé, développeur. État : **Implémenté**.

But

Réduire l'image de travail en **régions de couleur connexes**, sélectionnables et éditables, qui serviront de base à la vectorisation puis aux objets de broderie.

Algorithme

1. Les pixels d'alpha < 128 sont traités comme **fond**.
2. Les pixels opaques sont convertis en **CIELAB** (espace perceptuel) ;
3. un **k-means** déterministe (graine fixe) sur un échantillon de pixels calcule jusqu'à N couleurs représentatives ;
4. chaque pixel est affecté au centre Lab le plus proche ;
5. par couleur, les **composantes connexes** (4-connexité, `cv : :connectedComponents`) deviennent des **régions**. Deux régions de même couleur mais disjointes restent distinctes ;
6. les régions plus petites que la taille minimale sont **absorbées** par leur voisine majoritaire (nettoyage).

Identifiants stables

Chaque région a un `RegionId` = index de slot + 1. Les slots ne sont **jamais** réutilisés : un identifiant reste valide pour toute la vie de la segmentation (important pour l'undo/redo). La carte est stockée comme `labels[y*w+x]` (0 = fond) et `region_slots[]` (la région ou nul/opt si supprimée/fusionnée).

Éditions

Action	Fonction	Effet
Sélection	<code>region_at(x, y)</code>	Renvoie la région sous un pixel
Fusion	<code>merge_regions(keep, absorb)</code>	Réétiquette <code>absorb</code> en <code>keep</code>
Suppression	<code>remove_region(id)</code>	Renvoie les pixels au fond (pas de fusion)
Recoloration	<code>recolor_region(id, rgb)</code>	Change la couleur représentative
Carte	<code>render_map(highlight)</code>	Image RGBA (fond transparent, sélection éclaircie)

Note : « Supprimer » fait **disparaître** la région (retour au fond) et ne la fusionne pas avec une voisine — corrigé après un retour utilisateur (voir *Limitations*). Pour transférer des pixels à une autre région, utilisez la **fusion** explicite.

Undo/redo

Les commandes `SetSegmentationCommand`, `MergeRegionsCommand`, `RemoveRegionCommand` et `RecolorRegionCommand` mémorisent les **indices de pixels** réétiquetés (pas une copie de la carte entière), ce qui rend l'annulation exacte et peu coûteuse.

Erreurs

- Image entièrement transparente → erreur `UserInput`.
- Nombre de couleurs hors bornes → refus.

Implémentation associée

- `libs/segmentation/include/openstitch/segmentation/segmentation.hpp`
- `libs/segmentation/src/segmentation.cpp` — `segment`, `merge_regions`, `remove_region`, `recolor_region`, `render_map`.
- `libs/commands/.../project_commands.hpp` — commandes de segmentation.
- Tests : `tests/unit/segmentation/test_segmentation.cpp`, `tests/unit/commands/test_undo_stack.cpp`.

Vectorisation

Public : utilisateur avancé, développeur. État : **Implémenté** (édition de nœuds limitée au déplacement).

But

Transformer une région (masque de pixels) en **contours vectoriels propres** : contour extérieur et trous, en coordonnées physiques (μm), simplifiés et nettoyés.

Algorithme

1. Construction du **masque binaire** de la région.
2. Extraction des contours et trous par `cv::findContours` (mode `RETR_CCOMP`, approximation `CHAIN_APPROX_SIMPLE`) — algorithme de Suzuki-Abe.
3. Conversion des points en **coordonnées physiques** (μm , origine au centre de l'image, Y vers le haut).
4. **Simplification** Douglas-Peucker par contour, avec une tolérance **adaptative** bornée à `périmètre / 16` : les petits trous (mât, cordage traversant une voile) ne sont pas aplatis au point de disparaître.
5. **Nettoyage** par union Clipper2 (`clean_to_path_sets`) qui reconstruit la hiérarchie extérieur/trous (règle pair-impair) et corrige les auto-intersections. Une région en plusieurs morceaux produit plusieurs `PathSet`.

Résultat

Un `document::VectorObject` contient un ou plusieurs `geometry::PathSet` (`outer + holes`), garde la couleur de la région et un lien vers la `RegionId` d'origine.

Édition de nœuds

Les nœuds de l'objet sélectionné s'affichent (poignées de taille constante) et se **déplacent** à la souris ; chaque déplacement passe par `MoveNodeCommand` (annulable).

Limitation : l'**ajout/suppression de nœuds**, la **conversion anguleux/lisse** et l'édition de tangentes de Bézier ne sont **pas** exposés dans l'interface (le modèle les prévoit : `PathNode` porte `tan_in/tan_out` et un `NodeType`).

Robustesse

La correction de la tolérance de simplification (adaptative) évite un défaut observé : des trous trop simplifiés faisaient déborder les remplissages (voir *Tatami* et *Limitations*).

Implémentation associée

- `libs/vectorization/src/vectorize.cpp` — `vectorize_region`.
- `libs/geometry/src/simplify.cpp` — Douglas-Peucker, aire signée.
- `libs/geometry/src/clean.cpp` — `clean_to_path_sets` (Clipper2 encapsulé).
- `libs/document/include/openstitch/document/vector_object.hpp` — `VectorObject`, `NodeRef`.
- `libs/commands/.../project_commands.hpp` — `AddVectorObjectCommand`, `MoveNodeCommand`.
- Tests : `tests/unit/vectorization/test_vectorize.cpp`,
`tests/unit/geometry/test_simplify.cpp`, `test_clean.cpp`.

Objets de broderie

Public : utilisateur avancé, développeur. État : **Implémenté** (trois types).

Définition

Un document `::EmbroideryObject` porte l'**intention de couture** : un type de point et ses paramètres, une couleur de fil, un lien vers l'objet vectoriel source, un état de visibilité et de verrouillage. Ses **points ne sont pas stockés** dans l'objet : ils sont régénérés à la demande (séparation intention/points).

Le variant `StitchParams`

Le type de point est un `std::variant` à trois alternatives :

Objet	Params	Géométrie suivie	Sous-couche
Point de contour	<code>RunningStitchParams</code>	contours de l'objet vectoriel	non
Remplissage tatami	<code>TatamiParams</code>	régions pleines (avec trous)	prévue, non générée
Colonne satin	<code>SatinParams</code>	deux rails portés par l'objet	centrale (point droit)

Le satin est particulier : il **porte sa propre géométrie** (deux rails éditables), car une colonne ne se déduit pas d'un simple contour.

Tableau récapitulatif

Objet	Générateur	Paramètres clés	État
Running (simple/double/triple)	<code>run_stitch + apply_repeats</code>	<code>stitch_length</code> , <code>min_length</code> , <code>repeats</code>	Implémenté
Tatami	<code>fill_tatami</code>	<code>row_spacing</code> , <code>stitch_length</code> , <code>angle</code> , <code>inset</code> , <code>stagger</code>	Implémenté
Satin	<code>fill_satin</code>	<code>density</code> , <code>pull_compensation</code> , <code>center_underlay</code>	Implémenté

Création

Depuis l'interface (menu Broderie) sur un objet vectoriel sélectionné, ou en lot par la **numérisation automatique** qui choisit le type selon la forme :

- bande **fine** satinable → satin ;
- zone **large** → tatami ;
- petite forme ou satin impossible → **contour** cousu (point triple).

Régénération

`generate_sequence(project)` parcourt les objets **visibles** dans l'ordre, insère un changement de couleur entre deux objets de couleurs différentes, et appelle le générateur adapté à chaque type via `std::visit`. Le résultat est une `StitchSequence`.

Implémentation associée

- `libs/document/include/openstitch/document/embroidery_object.hpp` — `EmbroideryObject`, `StitchParams`, `RunningStitchParams`, `TatamiParams`, `SatinParams`.
- `libs/stitch_generation/src/generate.cpp` — `generate_sequence`, `dispatch`.
- `libs/autodigitize/src/autodigitize.cpp` — choix automatique du type.
- Tests : `tests/unit/stitch/test_generate.cpp`.

Génération de points — point droit et fondations

Public : utilisateur avancé, développeur. État : **Implémenté** (reconstruit sur des fondations de longueur d'arc).

Séquence — génération des points

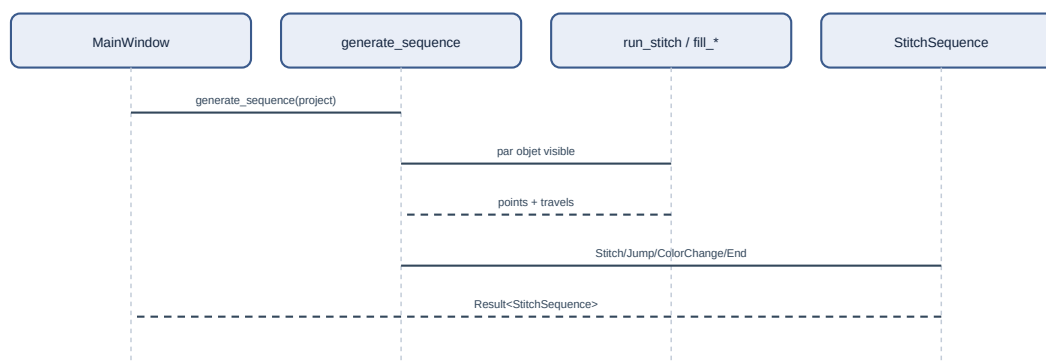


Figure — Génération de la séquence de points à partir du document.

Fondations géométriques

Avant tout type de point, deux primitives (dans `geometry`) :

- **Aplatissement de Bézier adaptatif** (`flatten`) : les segments à tangentes sont subdivisés récursivement (De Casteljau) jusqu'à ce que l'écart à la corde passe sous une tolérance. On n'échantillonne **jamais** une Bézier par pas uniforme du paramètre t .
- **Paramétrisation par longueur d'arc** (`cumulative_lengths`, `point_at_length`) et **ré-échantillonnage équilibré** (`resample_run`) : un tronçon de longueur L est divisé en $n = \text{ceil}(L / \text{cible})$ parts **égales**, donc chaque point est $\leq$ la longueur cible et il n'y a pas de segment résiduel minuscule.

Point droit (running stitch)

`run_stitch(path, config)` :

1. aplatit le chemin (courbes comprises) ;
2. détecte les **coins vifs** (angle de rotation $>$ seuil, $\sim 35^\circ$ par défaut) ;
3. découpe le chemin en tronçons entre coins ;
4. ré-échantillonne **chaque tronçon par longueur d'arc**.

Résultat : les coins sont des pénétrations **exactes** ; les portions lisses ont un espacement **régulier**. Un cercle finement facetté ne produit donc plus un point par facette, mais des points espacés de la longueur cible.

Le résultat est structuré : `RunningResult { points, warnings, stats }`. Il ne plante jamais et renvoie un avertissement (chemin vide, trop court, point trop long...).

Chemins fermés, sens, phase, départ

`RunningConfig` expose : `reverse` (sens), `phase` (décalage curviligne des boucles lisses) et `start` (point de départ projeté sur une boucle fermée). Une boucle fermée revient à son point de départ sans doublon minuscule.

Répétitions

`apply_repeat_mode` implémente des modes **explicites** :

Mode	Effet
<code>SinglePass</code>	un passage
<code>BackAndForth</code>	aller complet puis retour (termine au départ)
<code>BeanStitch</code>	chaque intervalle cousu n fois (impair) — point triple
<code>Backstitch</code>	progression avec recouvrement

Les mouvements nuls sont supprimés ; le nombre exact de traversées est testé.

Outil de diagnostic

`openstitch-cli stitchdebug --shape circle|line|corner|bezier|star|ring --length 3 --output-svg out.svg` génère les points sur une forme de référence, affiche les statistiques (points, longueur, segment min/max) et un SVG. Utilisé pour valider le moteur (voir *Tests*).

Implémentation associée

- `libs/geometry/src/polyline.cpp` — `flatten`, longueur d'arc, `resample_run`.
- `libs/stitch_generation/src/running_stitch.cpp` — `run_stitch`, `apply_repeat_mode`, `apply_repeats`, `sample_path`.
- `apps/cli/main.cpp` — sous-commande `stitchdebug`.
- Docs de conception : `docs/stitch-engine-audit.md`, `docs/stitch-engine-research.md`.
- Tests : `tests/unit/geometry/test_polyline.cpp`, `tests/unit/stitch/test_running_stitch.cpp`.

Colonne satin

Public : utilisateur avancé, développeur. État : **Implémenté** (base) ; auto-satin **partiel**.

Définition

Une colonne satin est un zigzag serré entre **deux rails** (bords gauche et droit) qui remplit une bande. Elle donne un effet lisse et brillant, adapté aux lettrages et aux bordures étroites.

Modèle

SatinParams porte rail_a et rail_b (deux geometry::Path), la densité (density), la compensation de tirage (pull_compensation), et l'activation d'une sous-couche centrale (center_underlay).

Génération

fill_satin(rail_a, rail_b, config) :

1. ré-échantillonne **les deux rails** par fraction d'abscisse curviligne (les rails peuvent avoir des longueurs et courbures différentes) ;
2. produit un **zigzag alterné** d'un bord à l'autre (L0, R0, L1, R1, ...) ;
3. la **densité** fixe le pas le long de la colonne ;
4. la **compensation de tirage** écarte les deux rails le long de leur médiatrice, pour compenser le resserrement du fil ;
5. la **sous-couche centrale** (optionnelle) est un point droit grossier sur l'axe, cousu avant le zigzag.

Le résultat (SatinResult) fournit les points de sous-couche, du satin, et la **largeur maximale** rencontrée (pour l'avertissement).

Colonne trop large

À la création, si la largeur dépasse le seuil recommandé, l'interface **prévient** et propose de préférer un remplissage tatami — la limite physique n'est jamais masquée.

Rails automatiques

rails_from_contour(contour) découpe un contour fermé en deux rails, coupé aux **deux sommets les plus éloignés** (les « bouts » de la colonne). Cela convient aux formes allongées ; l'auto-numérisation l'utilise pour proposer un satin.

Limitation : c'est une heuristique. Il n'y a **pas** d'axe médian (medial axis / straight skeleton), pas de barreaux de direction, pas de correspondance par sections, pas de gestion fine des points courts dans

les virages, ni de split stitch pour les colonnes très larges. Ces éléments (§12 de l'étude de cadrage) sont **prévus**. La densité est mesurée le long du rail, pas encore strictement perpendiculairement au fil.

Implémentation associée

- `libs/document/.../embroidery_object.hpp` — `SatinParams`.
- `libs/stitch_generation/include/openstitch/stitch_generation/satin.hpp`
- `libs/stitch_generation/src/satin.cpp` — `fill_satin`, `rails_from_contour`.
- `libs/stitch_generation/src/generate.cpp` — `generate_satin`.
- Tests : `tests/unit/stitch/test_satin.cpp`.

Remplissage tatami

Public : utilisateur avancé, développeur. État : **Implémenté** (routage par graphe) ; sous-couche tatami prévue.

Définition

Le tatami remplit une zone pleine par des **rangées parallèles** de points, avec un décalage régulier des pénétrations d'une rangée à l'autre pour éviter un sillon visible. C'est le remplissage de référence pour les surfaces larges.

Génération par balayage (scanline)

`fill_tatami(region, params)` :

1. la géométrie (extérieur + trous) est virtuellement **tournée** de `-angle` pour travailler avec des rangées horizontales ;
2. pour chaque rangée `y`, on calcule les **intersections** avec tous les contours et on forme les **segments intérieurs** par paires (règle pair-impair) — une rangée peut donc comporter **plusieurs segments** (formes concaves, trous) ;
3. sur chaque segment, les pénétrations sont posées sur une **grille** de pas `stitch_length`, décalée d'une rangée à l'autre selon `stagger` ;
4. la géométrie est retournée dans le repère d'origine.

Routage qui contourne les trous

C'est le point délicat. Les segments de rangée sont les **nœuds d'un graphe** : deux segments de rangées voisines qui **se chevauchent en x** sont adjacents (la bande entre eux est intérieure, donc une liaison ne peut pas traverser un trou — le trou aurait coupé la rangée). Un **parcours glouton** suit toujours une arête du graphe (couture qui reste dans la région) et ne fait un **déplacement** que vers un segment non adjacent.

Chaque point de remplissage est un `FillStitch { pos, travel }` : `travel = true` signale un déplacement (aiguille relevée). Ainsi **aucune couture ne traverse un trou ni ne sort de la région**.

Note : Ce comportement corrige un défaut signalé par un utilisateur (les remplissages débordaient sur les régions à trou). Vérification via `openstitch-cli stitchdebug --shape ring` : sur un anneau, 0 couture traverse le trou et le contournement ne coûte que ~2 déplacements.

Paramètres

Paramètre	Rôle
<code>row_spacing</code>	espacement entre rangées (densité)
<code>stitch_length</code>	longueur de point le long d'une rangée
<code>angle</code>	orientation des rangées
<code>inset</code>	retrait de bord (compensation de contour, offset Clapper)
<code>stagger</code>	nombre de rangées avant répétition de la phase des pénétrations

Le retrait de bord est appliqué **en amont** (offset intérieur du polygone) par `generate_tatami`.

Limitation : la **sous-couche tatami**, l'**underpath caché** (déplacements cousus sous la couche supérieure au lieu de sauts), les points d'**entrée/sortie** imposés et les motifs de phase avancés sont **prévus**, non implémentés.

Implémentation associée

- `libs/document/.../embroidery_object.hpp` — `TatamiParams`.
- `libs/stitch_generation/include/openstitch/stitch_generation/tatami.hpp` — `FillStitch`, `fill_tatami`.
- `libs/stitch_generation/src/tatami.cpp` — scanline + graphe de routage.
- `libs/stitch_generation/src/generate.cpp` — `generate_tatami`, `emit_fill`.
- `libs/geometry/src/offset.cpp` — `inset_path_set` (retrait de bord).
- Tests : `tests/unit/stitch/test_tatami.cpp` (invariant : aucune couture dans le trou), `tests/unit/geometry/test_offset.cpp`.

Retouche des points et de la géométrie

Public : utilisateur avancé, développeur.

Ce qui est éditable aujourd'hui

Niveau	Retouche disponible	État
Image	pile d'opérations non destructive	Implémenté
Segmentation	fusion, suppression, recoloration	Implémenté
Vecteurs	déplacement de nœuds	Implémenté
Objets de broderie	paramètres (longueur, densité, angle, type)	Implémenté
Ordre de couture	monter/descendre, verrouiller	Implémenté
Points générés	—	Non implémenté

Régénération vs édition manuelle

Les points sont **régénérés** à chaque modification du document (fonction pure). Il n'existe **pas** d'édition manuelle des points générés (déplacer un point, convertir un point en saut, ajouter une coupe). Le modèle prévoit cette distinction (états `Clean/Dirty/ManuallyEdited` décrits dans l'étude de cadrage) mais elle n'est pas exposée.

Cas particulier : une séquence **importée d'un DST** est traitée comme la vérité (le DST n'a pas d'objets), elle n'est donc pas régénérée tant qu'aucune image n'est chargée.

Édition de nœuds vectoriels

Sélectionnez un objet vectoriel : ses nœuds s'affichent en poignées. Un déplacement passe par `MoveNodeCommand` (annulable) et **régénère** les points de tout objet de broderie qui suit cet objet.

Limitation : l'ajout/suppression de nœuds, la conversion anguleux/lisse et l'édition de tangentes ne sont pas exposés (voir *Vectorisation*).

Implémentation associée

- `apps/desktop/node_handle.hpp` — poignée de nœud.
- `apps/desktop/main_window.cpp` — sélection d'objet, affichage des poignées.
- `libs/commands/.../project_commands.hpp` — `MoveNodeCommand`.

Palettes et fils

Public : utilisateur avancé, mainteneur. État : **Prévu / partiel**.

Ce qui existe

- Chaque objet de broderie porte une **couleur RGB** (`std::array<uint8_t, 3>`), reprise de la région d'origine.
- Un **changement de couleur** est inséré automatiquement entre deux objets consécutifs de couleurs différentes lors de la génération.
- La recoloration d'une région est disponible (choix d'une couleur libre).

Ce qui n'existe pas encore

Limitation : il n'y a **pas** de véritable **palette de fils** :

- pas de base de fils par fabricant/gamme/référence ;
- pas de recherche du fil le plus proche par distance perceptuelle ;
- pas d'association objet ↔ bobine/aiguille ;
- pas d'import/export de palette.

La bibliothèque `thread_palette` évoquée dans l'étude de cadrage **n'est pas présente** dans `libs/`. La couleur reste un simple RGB attaché à l'objet.

Conséquence pour le DST

Le format DST ne stocke de toute façon **pas** les couleurs réelles, seulement des arrêts « changement de fil ». L'ordre des couleurs est porté par le document `.osp`, pas par le DST (voir *Format DST*).

Implémentation associée

- `libs/document/.../embroidery_object.hpp` — champ `rgb`.
- `libs/stitch_generation/src/generate.cpp` — insertion des `ColorChange`.
- `libs/segmentation/src/segmentation.cpp` — `recolor_region`.
- *Information non déterminée* : aucune source `thread_palette` dans le dépôt.

Simulation de couture

Public : utilisateur. État : **Implémenté** (base).

But

Rejouer visuellement la couture pour repérer les sauts, l'ordre et la trajectoire de l'aiguille avant l'export.

Fonctionnement

La barre de simulation apparaît dès qu'une séquence de points existe. Elle offre :

- un bouton **Lecture / Pause** ;
- un **curseur** qui révèle la couture jusqu'à un index de point ;
- un **compteur** « point courant / total » ;
- un **repère d'aiguille** (cercle rouge) à la position courante.

Pendant la lecture, un minuteur (~60 pas/seconde) avance l'index proportionnellement à la taille du motif. Seule la **couche des points** est reconstruite à chaque image (l'image de fond et les vecteurs ne sont pas recalculés), pour rester fluide.

Limitation : la simulation est **point par point** via le curseur. Le réglage de vitesse explicite, l'avance objet-par-objet ou couleur-par-couleur, et l'affichage de l'objet actif ne sont **pas** exposés (le modèle en garde l'information : chaque `StitchCommand` porte son `ObjectId` source).

Implémentation associée

- `apps/desktop/main_window.cpp` — `buildSimulationToolbar`, `toggleSimulation`, `onSimTick`, `onSimSliderMoved`, `renderStitches`.
- `libs/stitch/include/openstitch/stitch/sequence.hpp` — `StitchSequence`, `StitchCommand` (avec source).

Analyse et validation

Public : utilisateur avancé, développeur. État : **Implémenté** (règles de base).

But

Détecter, avant l'export, les problèmes courants de broderie sans jamais appliquer de correction automatique silencieuse.

Règles détectées

`analyze(sequence, options)` renvoie une liste de `Finding` triés par gravité :

Catégorie	Condition (par défaut)	Gravité
<code>vide</code>	aucun point	Erreur
<code>point-court</code>	segment cousu < 0,5 mm	Avertissement
<code>point-long</code>	segment cousu > 7 mm	Avertissement
<code>saut-long</code>	déplacement > 30 mm	Avertissement
<code>hors-cadre</code>	point hors du cadre (si fourni)	Erreur
<code>trop-de-points</code>	> 100 000 points	Avertissement

Chaque problème porte une **gravité** (`Info/Warning/Error`), un **message**, une **localisation** et l'**objet** concerné. Un plafond par catégorie évite l'inondation ; le résultat est **déterministe**.

Dans l'interface

Analyse → **Analyser le motif** (F5) remplit le dock *Analyse* ; un double-clic sur un problème centre la vue sur sa localisation. Le cadre utilisé est le cadre courant (100 × 100 mm par défaut).

Limitation : l'analyse **spatiale de densité** (carte de chaleur, superposition de couches, satin trop large/étroit dédié, alignement excessif des pénétrations) et les **corrections automatiques proposées** sont **prévues**, non implémentées.

Implémentation associée

- `libs/stitch_analysis/include/openstitch/stitch_analysis/analyze.hpp` — `Finding`, `Severity`, `AnalysisOptions`.
- `libs/stitch_analysis/src/analyze.cpp` — `analyze`.
- `apps/desktop/main_window.cpp` — `buildAnalysisPanel`, `runAnalysis`.
- Tests : `tests/unit/stitch_analysis/test_analyze.cpp`.

Format DST (Tajima)

Public : utilisateur avancé, développeur, mainteneur. État : **Implémenté** (encodeur + décodeur maison, testés en aller-retour).

Enregistrement DST : 3 octets = déplacement (dx, dy) en unités de 0,1 mm

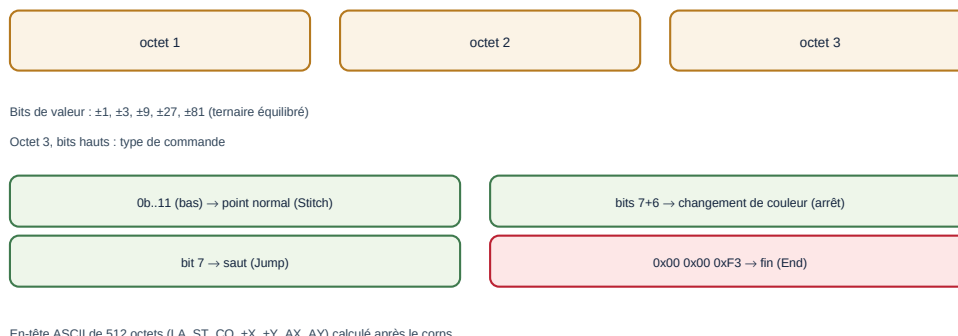


Figure — Structure d'un enregistrement DST de 3 octets et des commandes.

Rôle du format

Le DST (Tajima) est le format d'échange le plus répandu pour la broderie machine. C'est un fichier **binaire** contenant un en-tête ASCII puis une suite de **déplacements relatifs** de l'aiguille. Le module `formats` implémente son encodage et son décodage **sans dépendance externe** (le format est documenté publiquement ; la bibliothèque `libembroidery` n'a servi que de référence croisée).

Unités

Le pas du DST est **0,1 mm**. En interne, le logiciel travaille en **micromètres** ($1 \mu\text{m} = 1/1000 \text{ mm}$) ; la conversion vers le DST se fait par division par 100, et la quantification n'intervient **qu'à l'export**.

En-tête (512 octets ASCII)

Champs terminés par CR, complétés d'espaces jusqu'à 512 octets. Calculés **après** le corps (donc cohérents avec les octets réellement produits) :

Champ	Signification
LA:	nom du motif (16 caractères)
ST:	nombre d'enregistrements
CO:	nombre de changements de couleur
+X: -X: +Y: -Y:	étendue du motif (unités 0,1 mm)
AX: AY:	position finale relative au départ
MX: MY: PD:	multi-motif (valeurs par défaut)

Corps : enregistrements de 3 octets

Chaque enregistrement encode un déplacement (dx, dy) dans $[-121, +121]$ unités de 0,1 mm, par bits de valeurs $\pm 1, \pm 3, \pm 9, \pm 27, \pm 81$ (ternaire équilibré) répartis sur les trois octets. Les bits hauts de l'octet 3 donnent le **type** :

Commande interne	Encodage DST
Stitch (point)	bits bas 11, point normal
Jump (saut)	bit 7 de l'octet 3
ColorChange / Stop	bits 7 + 6 de l'octet 3
Trim (coupe)	convention : N sauts de delta nul (défaut 3)
End (fin)	0x00 0x00 0xF3

Encodage (sans dérive)

`encode_dst` quantifie les **positions absolues** au pas de 0,1 mm puis en dérive les deltas : $\text{delta}_i = \text{round}(\text{pos}_i/100) - \text{round}(\text{pos}_{i-1}/100)$. Cela borne l'erreur à $\pm 50 \mu\text{m}$ **sans dérive cumulative**. Tout déplacement dépassant ± 121 est **subdivisé** en sauts intermédiaires. La sortie est **déterministe**.

Décodage (tolérant)

`decode_dst` lit l'en-tête de façon **laxiste** (la vérité est le corps), décode chaque enregistrement, réinterprète une rafale de sauts nuls en Trim, et **ne plante jamais** : un fichier tronqué ou sans marqueur de fin renvoie une erreur claire.

Exemple interprété (motif minimal)

Un carré de 5 mm : après l'en-tête, une commande Jump amène au premier coin, puis quatre points `Stitch` de 5 mm parcourent les côtés, et l'enregistrement `0x00 0x00 0xF3` termine. Sur ce motif, `+X:` et `+Y:` valent 50 (5 mm en unités de 0,1 mm) et `co:` vaut 0.

Limites et informations perdues

Avertissement : Un DST **ne conserve pas** les objets éditables de haut niveau (contours, remplissages, colonnes satin, paramètres). Il ne stocke pas non plus les **couleurs réelles** (seulement des arrêts). Après un export DST, seul le **projet .osp** permet de retrouver la géométrie et les paramètres.

Tests

- **Aller-retour** : séquence → encode → decode → séquence' (mêmes types, positions à $\pm 50 \mu\text{m}$) — dont un test sur **tous les deltas** de -121 à $+121$.
- **Subdivision** d'un déplacement $> 12,1 \text{ mm}$.
- **Fichiers invalides** (vide, tronqué, sans fin) refusés proprement.
- **Déterminisme** octet à octet.

Implémentation associée

- `libs/formats/include/openstitch/formats/dst.hpp` — API.
- `libs/formats/src/dst.cpp` — `encode_dst`, `decode_dst`, `write_dst_file`, `read_dst_file`.
- `apps/cli/main.cpp` — `stats`, `dst2svg`.
- Tests : `tests/unit/formats/test_dst.cpp`.

Format de projet .osp

Public : développeur, mainteneur. État : **Implémenté**.

Nature

Un projet .osp est une **archive ZIP** (via minizip-ng) contenant :

Entrée	Contenu
project.json	tout le document, en JSON versionné
original.png	l'image source, encodée PNG (sans perte)
segmentation.u32	la carte des labels, en binaire little-endian (si présente)

Le JSON et le binaire sont séparés pour ne pas alourdir le JSON (la carte des labels peut faire plusieurs mégaoctets).

Contenu de project.json

- `schemaVersion` (entier) et un objet `document` ;
- `mmPerPx` (résolution de travail) et `objectIdLast` (compteur d'ids) ;
- `ops` : la pile d'opérations d'image (chaque opération porte son `type`) ;
- `segmentation` : dimensions et régions (id, rgb, nombre de pixels ; les labels sont dans `segmentation.u32`) ;
- `vectorObjects` : id, nom, couleur, visibilité, région source, `paths` (chemins avec nœuds et tangentes optionnelles) ;
- `embroideryObjects` : id, nom, couleur, visibilité, vecteur source, et `params` (variant `running` | `tatami` | `satin`).

Versionnement et validation

`schemaVersion` vaut actuellement **1**. À la lecture, une version inconnue ou supérieure est **refusée** proprement (`UnsupportedFormat`) ; un JSON invalide ou une carte de labels incohérente avec ses dimensions renvoie une erreur.

Sauvegarde atomique

`save_project` écrit d'abord un fichier temporaire `.osp.tmp` puis le **renomme** : un plantage pendant l'écriture ne corrompt jamais le fichier existant.

Aller-retour

Le test d'intégration vérifie que `save` puis `load` restitue exactement l'image (PNG sans perte), les opérations, les labels de segmentation, les tangentes de nœuds et les paramètres des trois types de points.

Limitation : la **sauvegarde automatique**, la **récupération après crash** et les **migrations** entre versions de schéma sont **prévues**, non implémentées. Le cache des points générés n'est pas stocké (il est recalculé au chargement).

Implémentation associée

- `libs/project_io/include/openstitch/project_io/project_io.hpp` — `save_project`, `load_project`, `kSchemaVersion`.
- `libs/project_io/src/json_serialize.cpp` — sérialisation du document.
- `libs/project_io/src/archive.cpp` — lecture/écriture ZIP (minizip-ng encapsulé).
- `libs/project_io/src/project_io.cpp` — orchestration, écriture atomique.
- Tests : `tests/unit/project_io/test_roundtrip.cpp`.

Architecture logicielle

Public : développeur, mainteneur.

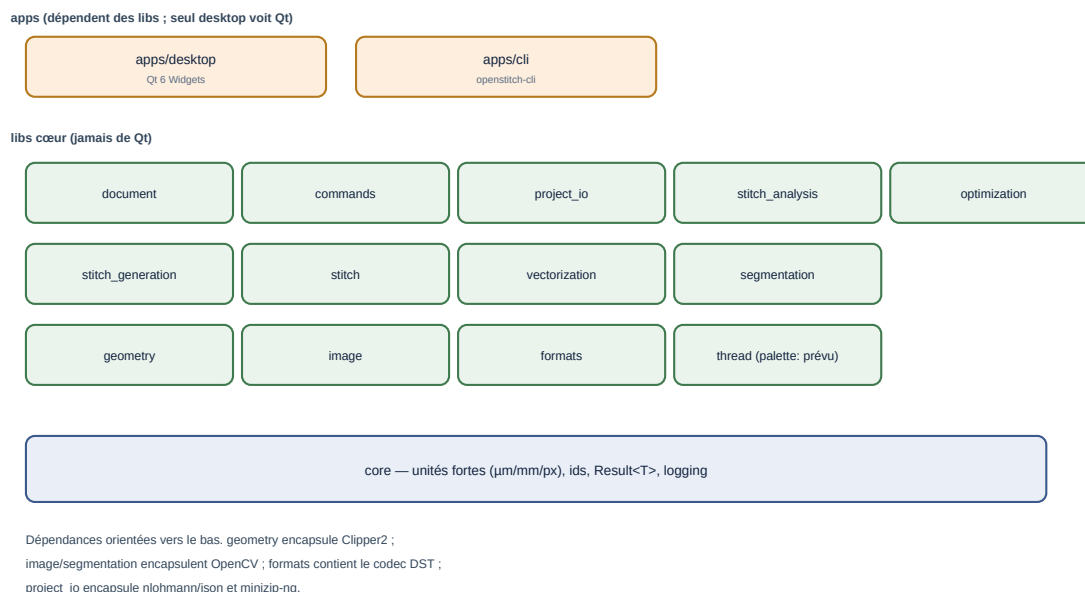


Figure — Les couches : applications, libs cœur, socle core.

Principes

1. **Le cœur ne dépend pas de Qt.** Seule apps/desktop voit Qt ; tout le reste se compile sans interface (vérifié par la CLI et par un build Linux du cœur).
2. **Dépendances orientées vers le bas** : apps → libs, et entre libs un graphe **acyclique**.
3. **Une lib = une cible CMake** openstitch::<nom>, avec ses tests.
4. **Encapsulation des tiers** : Clipper2 dans geometry, OpenCV dans image/segmentation/vectorization, le codec DST dans formats, json et minizip dans project_io.

Couches

- **Applications** : apps/desktop (Qt), apps/cli.
- **Métier / génération** : document, commands, project_io, stitch_generation, stitch, stitch_analysis, optimization, autodigitize, formats.
- **Traitement** : image, segmentation, vectorization, geometry.
- **Socle** : core (unités, ids, Result, logging).

Séparation cœur / interface

L'application ne contient **aucune logique métier** : elle câble des actions, affiche, et recalcule l'aperçu. Le chargement d'image, le placement physique, les transformations, la génération de points, l'analyse, l'ordre et les formats vivent tous dans les libs. La règle est vérifiable : la CLI utilise les mêmes modules sans Qt.

Modèle de document et rendu

Le `document::Project` est la source de vérité (voir *Modèle de données*). Les **points** ne sont pas stockés : ils sont recalculés par `generate_sequence`. Le rendu (côté desktop) est organisé en **deux couches** persistantes — base (image/vecteurs/régions) et points — pour ne reconstruire que le nécessaire, notamment pendant la simulation.

Commandes et undo/redo

Toute mutation du document passe par une **commande** (`ICommand`) empilée dans un `UndoStack`. C'est le patron *Command* : chaque commande sait s'appliquer et se révoquer exactement.

Tâches asynchrones

Limitation : il n'y a **pas** d'infrastructure de tâches asynchrones (pool de threads, annulation, progression) implémentée. Les traitements lourds (segmentation, remplissage) s'exécutent de façon **synchrone** (avec un curseur d'attente). Les fonctions de génération sont **pures** sur des instantanés, donc compatibles avec une exécution en arrière-plan future.

Diagrammes de séquence

- Import d'une image : voir *Introduction* et la figure du chapitre.
- Génération des points : voir *Génération de points*.
- Export DST : voir *Format DST*.

Séquence — import d'une image

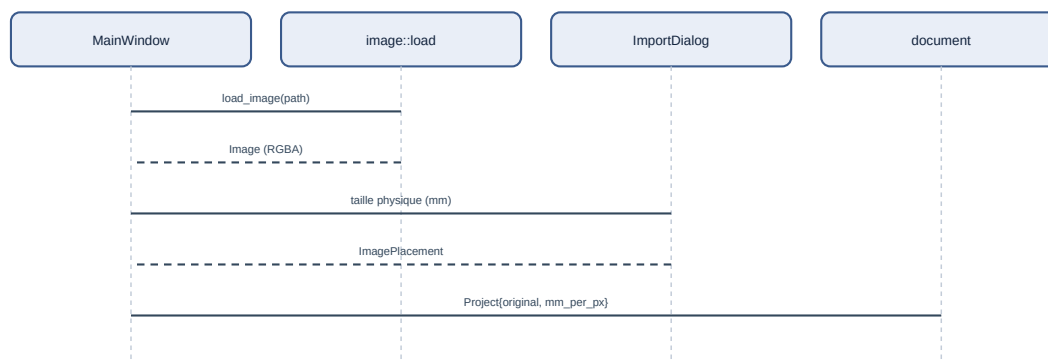


Figure — Import d'une image (du menu au document).

Implémentation associée

- `CMakeLists.txt` (racine), `libs/*/CMakeLists.txt` — cibles et dépendances.
- `apps/desktop/main_window.cpp` — câblage interface.
- `libs/commands/` — `ICommand`, `UndoStack`.
- `docs/phase0/02-architecture.md` — étude d'architecture d'origine.

Référence des modules

Public : développeur. Ce chapitre aide à trouver où modifier une fonctionnalité. Chaque module est une cible `openstitch::<nom>` sous `libs/`.

Tableau de synthèse

Module	Responsabilité	Dépendances	Tests
core	unités fortes, ids, Result, logging	—	tests/unit/core
geometry	chemins, simplification, booléens/offsets, longueur d'arc	core (+Clipper2)	tests/unit/geometry
image	chargement, prétraitement non destructif	core (+OpenCV)	tests/unit/image
segmentation	quantification CIELAB, régions connexes	core, image (+OpenCV)	tests/unit/segmentation
vectorization	régions → contours vectoriels	core, geometry, segmentation	tests/unit/vectorization
document	modèle métier (projet, objets)	core, geometry, image, segmentation	via commands/project_io
stitch	commandes machine, statistiques	core	tests/unit/stitch
stitch_generation	running / tatami / satin	core, geometry, stitch, document	tests/unit/stitch
stitch_analysis	règles de validation	core, stitch	tests/unit/stitch_analysis
optimization	ordre de couture	core	tests/unit/optimization
autodigitize	image → objets éditables	vectorization, stitch_generation	(intégration)
commands	undo/redo	document	tests/unit/commands
formats	codec DST, SVG diagnostic	core, stitch	tests/unit/formats
project_io	format .osp	core, document, image	tests/unit/project_io

Points d'extension

- **Ajouter un type de point** : nouvelle alternative de `StitchParams` (document), un générateur dans `stitch_generation`, une branche `std::visit` dans `generate.cpp`, un dialogue et une action dans `apps/desktop`.
- **Ajouter un format d'export** : une fonction dans `formats` (encapsulée derrière une interface interne), une sous-commande CLI et une action Fichier.
- **Ajouter une règle d'analyse** : une catégorie dans `analyze.cpp`.
- **Ajouter une opération d'image** : une alternative de `ImageOp` (image), un cas dans `apply_op`, une action menu.
- **Ajouter une commande annulable** : une classe `ICommand` dans `commands`.

Où trouver quoi (raccourci)

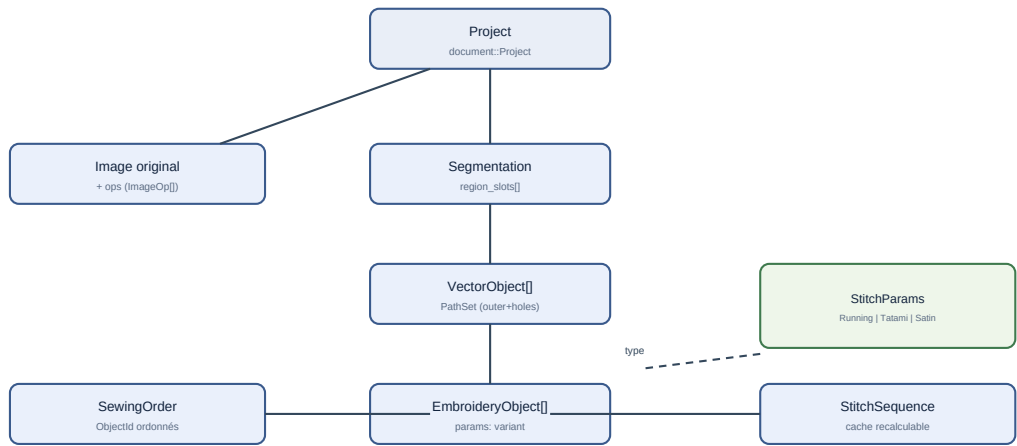
Je veux modifier...	Fichier
l'échantillonnage des points	<code>libs/stitch_generation/src/running_stitch.cpp</code> , <code>polyline.cpp</code>
le remplissage/routage tatami	<code>libs/stitch_generation/src/tatami.cpp</code>
la colonne satin	<code>libs/stitch_generation/src/satin.cpp</code>
l'encodage DST	<code>libs/formats/src/dst.cpp</code>
le format projet	<code>libs/project_io/src/</code>
les menus	<code>apps/desktop/main_window.cpp</code>
les unités	<code>libs/core/include/openstitch/core/units.hpp</code>

Implémentation associée

Voir les chapitres thématiques pour le détail de chaque module.

Modèle de données

Public : développeur.



La géométrie éditable (VectorObject) et les points générés (StitchSequence) sont séparés : régénérer ne détruit pas la source.

Figure — Relations principales du document.

Unités et coordonnées

Type	Fichier	Rôle	Unité
Micrometers	core/units.hpp	coordonnée interne (int32)	µm
Millimeters	core/units.hpp	affichage / paramètres UI	mm (double)
Pixels	core/units.hpp	domaine image	px (double)
Angle	core/units.hpp	angles	radians
Vec2um	core/units.hpp	point 2D	µm

Invariant : **aucune coordonnée n'est un double nu** ; l'unité interne est le micromètre entier (déterminisme, pas de dérive), Y vers le haut. La conversion vers/depuis les pixels exige une résolution explicite (mm/px).

Identifiants

ObjectId, RegionId, ColorId, ThreadId sont des types **forts distincts** (Id<Tag>), générés par un IdGenerator<T> monotone (jamais réutilisés). Fichier : core/ids.hpp.

Document

`document::Project` (fichier `document/project.hpp`) contient :

Champ	Type	Rôle
<code>original</code>	<code>image::Image</code>	image source intacte
<code>mm_per_px</code>	<code>Millimeters</code>	résolution de travail
<code>ops</code>	<code>vector<ImageOp></code>	pile de prétraitements
<code>segmentation</code>	<code>optional<Segmentation></code>	carte de régions
<code>vector_objects</code>	<code>vector<VectorObject></code>	géométrie éditable
<code>embroidery_objects</code>	<code>vector<EmbroideryObject></code>	objets de broderie (ordre = ordre de couture)
<code>object_ids</code>	<code>IdGenerator<ObjectId></code>	compteur partagé

Types géométriques

- `geometry::PathNode` : `pos` (`Vec2um`), `type` (`Corner/Smooth`), `tan_in/tan_out` optionnelles.
- `geometry::Path` : `nœuds` + `closed`.
- `geometry::PathSet` : `outer` + `holes`.
- `geometry::Polyline` : polyligne aplatie (travail par longueur d'arc).

Points et commandes

- `stitch::CommandType` : `Stitch`, `Jump`, `Trim`, `ColorChange`, `Stop`, `End`.
- `stitch::StitchCommand` : `pos` (`Vec2um` absolue), `type`, `source` (`ObjectId`).
- `stitch::StitchSequence` : `vector<StitchCommand>`.
- `stitch::StitchStats` : compteurs, longueur de fil, bornes.

Sérialisation

Voir *Format de projet* : tout ce qui précède se sérialise en JSON (+ PNG + binaire), sauf les points générés (recalculés).

Éléments non présents dans le modèle

Limitation : `profil machine`, `cadre` paramétrable riche, `fil/palette`, `avertissement` persistant et `commande undo` sérialisée ne sont pas (ou peu) présents. Le `canvas` existe mais reste minimal (taille du `cadre`).

Implémentation associée

- `libs/core/include/openstitch/core/{units,ids,error}.hpp`
- `libs/document/include/openstitch/document/{project,vector_object,embroidery_object,canvas}.hpp`
- `libs/geometry/include/openstitch/geometry/{path,polyline}.hpp`
- `libs/stitch/include/openstitch/stitch/sequence.hpp`

Algorithmes

Public : développeur. Ce chapitre décrit les algorithmes **réellement** utilisés, avec un pseudo-code reflétant l'implémentation (et non une copie du C++).

Quantification (segmentation)

- **Objectif** : réduire l'image à N couleurs perceptuellement cohérentes.
- **Principe** : conversion RGB→CIELAB, k-means déterministe sur un échantillon, affectation au centre le plus proche.
- **Complexité** : $\sim O(\text{pixels} \cdot N)$ pour l'affectation.
- **Cas limites** : image entièrement transparente (refus), N hors bornes.

```
segment(image, N):
    opaques = pixels d'alpha >= 128 en Lab
    centres = kmeans(échantillon(opaques), N, graine fixe)
    pour chaque pixel opaque: label = argmin_c ||Lab(pixel) - centres[c]||
    régions = composantes_connexes(label) # 4-connexité, par couleur
    absorber les régions < taille_min dans la voisine majoritaire
```

Extraction et simplification de contours

- **Contours** : Suzuki-Abe (cv::findContours, RETR_CCOMP).
- **Simplification** : Douglas-Peucker, tolérance **adaptative** ($\leq \text{périmètre}/16$) pour préserver les petits trous.
- **Nettoyage** : union Clipper2 (règle pair-impair) → hiérarchie extérieur/trous.

Aplatissement de Bézier + longueur d'arc

```
flatten(path, tol):
    pour chaque segment à tangentes: subdiviser (De Casteljaeu) tant que
        distance(contrôles, corde) > tol
    fusionner les points identiques

resample_run(points, cible):
    L = longueur(points)
    n = max(1, ceil(L / cible)) # parts égales, <= cible, sans résidu
    renvoyer points aux abscisses i*L/n, i=0..n
```

Running stitch

```
run_stitch(path, cfg):
    poly = flatten(path, cfg.tol)
    coins = sommets d'angle de rotation > seuil (+ extrémités si ouvert)
    pour chaque tronçon entre coins: concat(resample_run(tronçon, cible))
```

- **Complexité** : $O(\text{points})$.
- **Cas limites** : chemin vide/trop court → avertissement, pas de crash.

Tatami (scanline + routage)

```
fill_tatami(région, params):
    tourner la géométrie de -angle
    pour chaque rangée y: segments = intervalles intérieurs (pair-impair)
    graphe: segments de rangées voisines qui se chevauchent en x sont adjacents
    parcours glouton: suivre une arête (couture) ; sinon sauter (déplacement)
    retourner dans le repère d'origine
```

- **Garantie** : aucune couture ne traverse un trou (adjacence $\Rightarrow$ bande intérieure).
- **Complexité** : $\sim O(\text{segments}^2)$ au pire pour l'adjacence locale par rangée (voisines uniquement), acceptable aux tailles usuelles.

Satin

```
fill_satin(railA, railB, cfg):
    pour u de 0 à 1 par pas dérivé de la densité:
        L = point(railA, u*longueurA) ; R = point(railB, u*longueurB)
        appliquer compensation (écarter L et R le long de LR)
        émettre zigzag alterné L,R
    sous-couche centrale = point droit sur (L+R)/2
```

Encodage / décodage DST

```
encode(seq):
    pour chaque commande: delta = round(pos/100) - round(prev/100)
        subdiviser si |delta| > 121 (sauts)
        émettre 3 octets (ternaire équilibré + bits de type)
    en-tête calculé après le corps
```

- **Propriété** : déterministe, erreur $\leq 50 \mu\text{m}$, sans dérive.

Ordre de couture

```
optimize_order(items, stratégie):
    items libres (non verrouillés) réarrangés:
        ByColor: regrouper par couleur (ordre d'apparition)
        ByProximity: plus proche voisin
        ColorThenProximity: groupes de couleur puis proximité
    réinsérer dans les emplacements libres, verrous à leur place
    coût = distance de déplacement + 50000 * changements de couleur
```

Implémentation associée

Voir chaque chapitre thématique et `docs/stitch-engine-research.md` pour les références (Douglas-Peucker, scanline fill, k-means, ré-échantillonnage par longueur d'arc).

Compilation et développement

Public : développeur, mainteneur.

Chaîne de build

- **CMake** (≥ 3.27) avec `CMakePresets.json`.
- **MSVC** (Visual Studio 2022+) sous Windows ; générateur par défaut du preset.
- **vcpkg** en mode manifeste (`vcpkg.json`, baseline verrouillée).
- **Qt 6.8** hors `vcpkg` (variable `QT_ROOT`).

Presets

Preset	Cible
<code>msvc</code> (configure)	Windows, application complète
<code>msvc-debug</code> / <code>msvc-release</code> (build/test)	Debug / Release
<code>linux-core</code>	Cœur + CLI sans Qt (garde-fou de portabilité)

Commandes

```
$env:VCPKG_ROOT = "C:\dev\vcpkg"
$env:QT_ROOT    = "C:\Qt\6.8.3\msvc2022_64"

cmake --preset msvc          # configuration (premier run long : vcpkg)
cmake --build --preset msvc-debug # compilation Debug
ctest --preset msvc-debug     # tests
cmake --build --preset msvc-release # Release
```

Options CMake

- `OPENSTITCH_BUILD_DESKTOP` (ON) — construit l'application Qt.
- `OPENSTITCH_BUILD_TESTS` (ON) — construit les tests (Catch2).
- Cible d'interface `openstitch::warnings : /W4 /permissive- /utf-8` (MSVC), `-Wall -Wextra -Wconversion` (GCC/Clang), liée en PRIVATE par chaque cible.

Déploiement des DLL (Windows)

L'application lie dynamiquement Qt et OpenCV. `windeployqt` copie les DLL Qt à côté de l'exécutable (étape `POST_BUILD`) ; `vcpkg` copie ses propres DLL.

Linux (cœur uniquement)

Le preset `linux-core` compile les libs cœur et la CLI **sans Qt** — c'est un garde-fou pour éviter toute dépendance Windows/Qt involontaire dans le cœur. L'interface graphique n'est pas construite sous Linux à ce stade.

Erreurs fréquentes

- « Qt6 introuvable » → vérifier `QT_ROOT`.
- Erreur de toolchain `vcpkg` → vérifier `VCPKG_ROOT` et le bootstrap.
- Compilation OpenCV très longue → normal au premier configure (cache ensuite).
- Édition de lien de l'exé échouée (`LNK1168`) → l'application est en cours d'exécution ; la fermer.

Intégration continue

Un workflow GitHub Actions (`.github/workflows/ci.yml`) est présent : build MSVC Debug/Release + tests + artefacts, build Linux du cœur, et vérification du formatage. *Information non déterminée* : aucun dépôt distant n'étant déclaré, la CI n'a pas encore été exécutée.

Implémentation associée

- `CMakeLists.txt`, `CMakePresets.json`, `vcpkg.json`.
- `.github/workflows/ci.yml`, `.clang-format`.
- `docs/build-windows.md`.

Guide du développeur

Public : développeur contributeur.

Conventions

- **C++23**, RAII, `std::optional`/`std::variant`/`std::span`/`std::filesystem`, `std::expected` (via `Result<T>`).
- **Types forts** pour les unités : jamais de coordonnée en `double` nu.
- **Const-correctness**, warnings élevés (`/W4` /permissive-).
- **Formatage** : `.clang-format` (base LLVM, 4 espaces, 100 colonnes).
- **En-tête de licence** : `// SPDX-License-Identifier: Apache-2.0`.

Pièges connus (spécifiques à ce dépôt)

Avertissement : Quelques écueils rencontrés lors du développement :

- les **noms de tests Catch2** doivent être en **ASCII** (les accents ne survivent pas à l'encodage console Windows lors de la découverte des tests) ;
- l'en-tête `spdlog` est `stdout_color_sinks.h` ;
- un membre nommé `slots` est interdit (macro Qt) — utiliser un autre nom ;
- déclarer les classes Qt en **forward declaration au scope global** ;
- pas de `M_PI` sous MSVC → utiliser `std::numbers::pi` (`<numbers>`) ;
- cibles `vcpkg` exactes : `Clipper2::Clipper2`, `MINIZIP::minizip-ng`.

Ajouter une fonctionnalité (recettes)

- **Nouveau type de point** : voir *Référence des modules* → *Points d'extension*.
- **Nouvelle opération d'image** : une alternative de `ImageOp`, un cas dans `apply_op`, un test, une action menu.
- **Nouvelle commande annulable** : dériver `ICommand` (`apply/revert/name`), l'exécuter via `UndoStack::execute`. Toute mutation du document doit passer par une commande.

Undo/redo

`UndoStack` conserve deux piles (`undo/redo`). Une nouvelle commande vide la pile `redo`. Les commandes de segmentation mémorisent les **indices** de pixels modifiés pour une annulation exacte et légère.

Gestion des unités

Toujours convertir aux frontières : pixels→ μm à l'import (résolution explicite), μm →mm pour l'affichage, μm →0,1 mm à l'export DST. Les algorithmes internes peuvent utiliser des `double`, mais les entrées/sorties des modules sont en types forts.

Multithreading

Il n'y a pas encore d'infrastructure asynchrone. Écrire les nouveaux traitements comme des **fonctions pures** sur des instantanés, pour rester compatible avec un futur passage en tâche de fond.

Implémentation associée

- `.clang-format`, `CMakeLists.txt` (cible `openstitch::warnings`).
- `libs/commands/` — `ICommand`, `UndoStack`, commandes du projet.
- `docs/phase0/` — décisions d'architecture (ADR).

Tests

Public : développeur, mainteneur.

Structure

- **Framework** : Catch2 v3 (via vcpkg), intégré à CTest.
- **Tests unitaires** : tests/unit/<lib>/test_*.cpp (un exécutable par lib).
- **Test d'intégration** : tests/integration/test_pipeline.cpp (chaîne complète).
- **Golden (SVG de diagnostic)** : tests/golden/stitch-generation/.
- Total au dernier passage : **139 tests**, 100 % réussis (Debug et Release).

Exécution

```
ctest --preset msvc-debug          # tous les tests
ctest --preset msvc-debug -R tatami # filtre par nom
build\msvc\tests\unit\stitch\Debug\test_stitch.exe "[nom]" # un exécutable
```

Correspondance modules ↔ tests

Module	Tests
core	tests/unit/core/test_units.cpp, test_ids.cpp
geometry	test_simplify.cpp, test_clean.cpp, test_offset.cpp, test_polyline.cpp
image	test_image_load.cpp, test_ops.cpp
segmentation	test_segmentation.cpp
vectorization	test_vectorize.cpp
stitch / stitch_generation	test_stats.cpp, test_running_stitch.cpp, test_generate.cpp, test_tatami.cpp, test_satin.cpp
stitch_analysis	test_analyze.cpp
optimization	test_order.cpp
commands	test_undo_stack.cpp
formats	test_dst.cpp, test_svg.cpp
project_io	test_roundtrip.cpp
intégration	tests/integration/test_pipeline.cpp

Types de vérifications notables

- **DST aller-retour** sur tous les deltas de -121 à $+121$, déterminisme octet à octet.
- **Tatami** : invariant « aucune couture ne traverse le trou » d'un anneau, peu de déplacements.
- **Running** : espacement par longueur d'arc (cercle), coins préservés, courbes Bézier suivies, résultats déterministes.
- **Undo/redo** : « undo total = état initial », restauration exacte des labels.
- **Projet** : aller-retour complet (image, ops, segmentation, tangentes, params).

Ajouter un test

Ajoutez un `TEST_CASE` (nom **ASCII**) dans le `test_*.cpp` du module, ou un nouveau fichier référencé dans le `CMakeLists.txt` du dossier de tests. Les golden SVG ne sont **jamais** réécrits par les tests : ils se régénèrent explicitement via `openstitch-cli stitchdebug`.

Implémentation associée

- `tests/` (arborescence complète), `tests/unit/*/CMakeLists.txt`.
- `CMakeLists.txt` (racine) — `enable_testing`, `Catch`.

Guide de contribution

Public : contributeur.

Flux

1. Cloner le dépôt et créer une **branche** dédiée (le développement se fait sur main en local ; ne pas committer directement sans branche pour une fonctionnalité).
2. Compiler et exécuter les tests (`ctest --preset msvc-debug`).
3. Ajouter/mettre à jour les **tests** couvrant la modification.
4. Formater (`clang-format`) et vérifier les warnings.
5. Rédiger un message de commit clair (voir ci-dessous).

Messages de commit

Le dépôt utilise des messages descriptifs en français, terminés par une ligne de co-auteur. Exemple observé :

```
Corrige le débordement des remplissages hors des regions
<description détaillée>
Co-Authored-By: ...
```

Style C++

- Respecter `clang-format` et les warnings élevés.
- Types forts pour les unités ; pas de logique métier dans les widgets Qt.
- Toute mutation du document via une **commande** (undo/redo).
- En-tête SPDX Apache-2.0 en tête de fichier.

Ajouter une dépendance

- La déclarer dans `vcpkg.json` (ou documenter l'installation si hors `vcpkg`).
- **Vérifier la licence** : compatible Apache-2.0, permissive de préférence. **Aucun code GPL** ne peut être copié dans ce dépôt (voir *Licences*).
- L'encapsuler derrière une interface interne (ne pas exposer ses types).
- La documenter dans `THIRD_PARTY_LICENSES.md`.

Signaler un bug / proposer une fonctionnalité

Information non déterminée dans le dépôt : aucun gestionnaire d'issues public n'est configuré (dépôt local). En interne, documentez le problème avec un cas minimal reproductible et, si possible, un test qui échoue.

Documentation

Toute fonctionnalité visible doit être reflétée dans `docs/source/` puis le PDF régénéré (voir `docs/README.md`). Les affirmations doivent rester **vérifiables** dans le code (section « Implémentation associée »).

Implémentation associée

- `.clang-format`, `THIRD_PARTY_LICENSES.md`, `LICENSE`.
- `docs/README.md` — régénération de la documentation.

Dépannage

Public : utilisateur, développeur. Pour chaque problème : symptôme, cause probable, diagnostic, solution.

Compilation

Symptôme	Cause probable	Solution
« Qt6 introuvable »	QT_ROOT absent/incorrect	Pointer sur <code>... \msvc2022_64</code> , reconfigurer
Erreur toolchain vcpkg	VCPKG_ROOT absent, bootstrap manquant	Définir la variable, relancer <code>bootstrap-vcpkg.bat</code>
DLL manquante au lancement	déploiement Qt/OpenCV incomplet	Lancer depuis le dossier Debug/Release (windeployqt a copié les DLL)
LNK1168 à l'édition de lien	l'application tourne encore	Fermer <code>openstitch.exe</code> puis recompiler
OpenCV très longue à compiler	premier configure vcpkg	Normal ; les runs suivants utilisent le cache

Usage

Symptôme	Cause probable	Diagnostic	Solution
Image non chargée	format/fichier corrompu	<code>openstitch-cli info fichier</code>	Réexporter l'image en PNG
Segmentation « incorrecte »	trop/pou de couleurs, petites régions	carte des régions	Ajuster N et la taille min, fusionner/supprimer
Aucun point généré	objet sans géométrie, tous invisibles	barre d'état / Statistiques	Vérifier qu'un objet vectoriel source existe et est visible
Satin impossible	rails non constructibles / forme non allongée	message à la création	Préférer un tatami
Remplissage qui déborde	trou perdu à la vectorisation	zoom sur la zone	Corrigé : tolérance adaptative + routage ; re-numériser
Motif hors cadre	taille physique trop grande	règles / cadre rouge	Réduire la taille à l'import, ou déplacer
Points trop longs	longueur de point élevée	Analyse (F5)	Réduire <code>stitch_length</code>
Export DST refusé	séquence vide	message	Générer des points d'abord
Fichier DST « vide »	aucun objet visible	<code>openstitch-cli stats</code>	Vérifier les objets et l'ordre
Couleurs incorrectes	pas de palette de fils	—	Fonctionnalité prévue ; RGB par objet en attendant
Projet impossible à ouvrir	<code>.osp</code> corrompu / version inconnue	message d'erreur	Vérifier le fichier ; version de schéma supportée = 1
Performances faibles	très gros motif	—	Corrigé : rendu deux couches, pastilles limitées
Crash	entrée inattendue	—	Aucune entrée utilisateur ne devrait faire planter ; signaler avec un cas minimal

Logs

Le logger (spdlog) écrit sur la **sortie standard/erreur** de la console. Lancer la CLI ou l'application depuis un terminal pour voir les messages. *Information non déterminée* : aucun fichier de log persistant n'est écrit par défaut.

Implémentation associée

- `libs/core/src/log.cpp` — initialisation du logger.
- `libs/*/src/*` — les erreurs renvoyées via `Result<T>` avec un message montrable.

Limitations et état des fonctionnalités

Public : tous. Ce chapitre distingue explicitement l'état de chaque fonctionnalité, vérifié dans le code.

Tableau des fonctionnalités

Fonctionnalité	État	Interface	Module	Tests	Limitations
Import PNG/JPEG/BMP/TIFF	Implémenté	Fichier	image	oui	—
Prétraitement non destructif	Implémenté	Image	image	oui	pas de resize
Segmentation CIELAB + régions	Implémenté	Segmentation	segmentation	oui	4-connexité
Édition de régions	Implémenté	Segmentation	segmentation	oui	—
Vectorisation	Implémenté	Segmentation	vectorization	oui	—
Édition de nœuds	Partiel	canevas	document	—	déplacement seul
Point droit/double/triple	Implémenté	Broderie	stitch_generation	oui	—
Tatami	Implémenté	Broderie	stitch_generation	oui	pas de sous-couche, pas d'underpath caché
Satin	Implémenté	Broderie	stitch_generation	oui	pas d'axe médian ni barreaux
Auto-numérisation	Implémenté	Segmentation	autodigitize	(intégration)	heuristique de type
Ordre de couture	Implémenté	dock	optimization	oui	2-opt non implémenté
Analyse	Implémenté	Analyse	stitch_analysis	oui	pas de carte de densité
Simulation	Implémenté	barre	desktop	—	pas de réglage de vitesse
Export/Import DST	Implémenté	Fichier/CLI	formats	oui	limites du format
Export SVG diagnostic	Implémenté	CLI	formats	oui	—
Format projet .osp	Implémenté	Fichier	project_io	oui	pas d'autosave/migration
Palette de fils	Non implémenté	—	(thread_palette absent)	—	RGB par objet uniquement
Édition manuelle des points	Non implémenté	—	—	—	régénération uniquement
Remplissages courbe/radial/spirale	Non implémenté	—	—	—	prévus
Profil machine/cadres avancés	Non implémenté	—	—	—	cadre fixe 100×100 mm
Compensation directionnelle	Partiel	—	—	—	satin uniquement
Tâches asynchrones	Non implémenté	—	—	—	traitements synchrones

Dette technique connue

- `README.md` mentionne « 121 tests » alors que le compte réel est 139 (jalon antérieur) — corrigé dans la présente documentation.
- Le satin et le tatami restent des versions de base ; la refonte du moteur de points (documentée dans `docs/stitch-engine-audit.md`) a traité les fondations et le running stitch ; satin/tatami avancés restent à reprendre.
- Aucune validation sur machine à broder réelle.

Statut de validation

Le logiciel est **validé numériquement** (139 tests) et **visuellement** (SVG de diagnostic, aperçu). Il n'est **pas** validé sur simulateur tiers ni physiquement. Ne pas considérer les résultats comme prêts pour la production sans essais machine.

Implémentation associée

Voir chaque chapitre thématique et l'annexe *Audit du dépôt*.

Roadmap

Public : mainteneur, contributeur. Cette roadmap est construite à partir du code existant, des documents de conception (docs/phase0/, docs/stitch-engine-*.md) et des fonctionnalités manquantes identifiables. **Aucune date n'est annoncée.**

Court terme (dette et robustesse)

- Refonte avancée du **satin** (§12 de l'étude) : correspondance par sections, barreaux de direction, densité perpendiculaire au fil, gestion des points courts dans les virages, split stitch pour les colonnes larges, terminaisons.
- Refonte avancée du **tatami** (§15) : underpath **caché** au lieu de sauts, points d'entrée/sortie imposés, sous-couches, motifs de phase.
- Séparer la **normalisation machine** de l'encodeur DST (préparer PES/JEF/EXP).

Moyen terme (fonctionnalités)

- **Palette de fils** (thread_palette) : base fabricants, distance perceptuelle, association objet↔bobine, import/export.
- **Édition manuelle des points** (déplacer, convertir en saut, ajouter une coupe) avec états Clean/Dirty/ManuallyEdited.
- **Sous-couches** paramétrables (contour, zigzag) pour satin et tatami.
- **Compensation** directionnelle et profils tissu/stabilisateur/fil.
- **Édition de nœuds** complète (ajout/suppression, tangentes, anguleux/lisse).

Long terme (avancé)

- **Auto-satin** par axe médian / straight skeleton et décomposition en colonnes.
- **Remplissages avancés** : concentrique, spirale, radial, guidé (champ de direction), motifs.
- **Autres formats** : PES, JEF, EXP, VP3.
- **Tâches asynchrones** (annulation, progression) et **carte de densité**.
- **Profils machine/cadres**, sauvegarde automatique, migrations de projet.
- Portage **macOS**, distribution de binaires, publication publique.

Sources

- docs/phase0/08-roadmap-adr.md — roadmap d'origine (13 phases).
- docs/stitch-engine-audit.md — défauts et priorités du moteur de points.

Implémentation associée

Les fonctionnalités « Non implémenté » du chapitre *Limitations* constituent la liste de travail. Chacune indique le module cible.

Glossaire

Terme français, terme anglais utilisé dans le code, définition, et type/classe associé lorsqu'il existe.

Français	Anglais (code)	Définition	Type / fichier
Point	stitch	Pénétration d'aiguille cousant le fil	<code>CommandType::Stitch</code>
Saut	jump	Déplacement de l'aiguille sans couture	<code>CommandType::Jump</code>
Coupe-fil	trim	Coupe du fil	<code>CommandType::Trim</code>
Changement de couleur	color change	Arrêt pour changer de fil	<code>CommandType::ColorChange</code>
Arrêt	stop	Arrêt machine	<code>CommandType::Stop</code>
Fin	end	Fin du motif	<code>CommandType::End</code>
Point droit / courant	running stitch	Points le long d'un chemin	<code>run_stitch</code>
Point triple	bean stitch	Chaque segment cousu 3×	<code>RepeatMode::BeanStitch</code>
Point arrière	backstitch	Progression avec recouvrement	<code>RepeatMode::Backstitch</code>
Remplissage	fill	Couture d'une surface	<code>fill_tatami</code>
Tatami	tatami	Remplissage par rangées parallèles	<code>TatamiParams</code>
Colonne satin	satin column	Zigzag entre deux rails	<code>SatinParams, fill_satin</code>
Rail	rail	Bord d'une colonne satin	<code>SatinParams::rail_a/b</code>
Sous-couche	underlay	Couche cousue avant la couche visible	<code>center_underlay</code>
Compensation de tirage	pull compensation	Élargissement pour compenser la traction	<code>pull_compensation</code>
Densité	density / row_spacing	Espacement des fils/rangées	<code>density, row_spacing</code>
Longueur de point	stitch length	Longueur d'un point	<code>stitch_length</code>
Angle	angle	Orientation du remplissage	<code>Angle</code>
Contour	contour / outer	Bord d'une région	<code>PathSet::outer</code>
Trou	hole	Contour intérieur	<code>PathSet::holes</code>
Chemin	path	Polyligne/courbe	<code>geometry::Path</code>
Région	region	Zone de couleur connexe	<code>segmentation::Region</code>
Objet vectoriel	vector object	Contours éditables	<code>document::VectorObject</code>
Objet de broderie	embroidery object	Intention de couture	<code>document::EmbroideryObject</code>
Séquence de points	stitch sequence	Commandes machine	<code>stitch::StitchSequence</code>
Cadre / tambour	hoop / canvas	Zone de broderie	<code>document::Canvas</code>
Palette	palette	Ensemble de couleurs/fils	<i>(prévu)</i>
Fil	thread	Fil de broderie	<i>(prévu)</i>
Ordre de couture	sewing order	Ordre des objets	<code>optimization, SewingOrder</code>
Micromètre	micrometer	Unité interne (1/1000 mm)	<code>Micrometers</code>
Déplacement (interne)	travel	Liaison non cousue d'un remplissage	<code>FillStitch::travel</code>

Licences

Public : mainteneur, contributeur.

Licence du logiciel

OpenStitch Studio est distribué sous **Apache-2.0** (permissive, avec clause de brevets). Fichier : `LICENSE`. Chaque fichier source porte `// SPDX-License-Identifier: Apache-2.0`.

Conséquence importante : le code sous licence **copyleft** (GPL, notamment Ink/Stitch) **ne peut pas** être copié, traduit ou adapté dans ce dépôt sans faire basculer le projet sous GPL. Seuls sont réutilisables les **algorithmes publiés** (méthodes classiques non protégeables) et les **bibliothèques permissives**. Voir `docs/stitch-engine-research.md`.

Dépendances externes

Bibliothèque	Version	Rôle	Licence	Intégration	URL
fmt	via vcpkg	formatage	MIT	vcpkg	github.com/fmtlib/fmt
spdlog	via vcpkg	journalisation	MIT	vcpkg	github.com/gabime/spdlog
CLI11	via vcpkg	args CLI	BSD-3-Clause	vcpkg	github.com/CLIUtils/CLI11
OpenCV (core/imgproc/imgcodecs)	via vcpkg	image	Apache-2.0	vcpkg	opencv.org
libpng / libjpeg-turbo / libtiff / zlib	transitives	codecs	zlib / IJG / libtiff / zlib	via OpenCV	—
Clipper2	via vcpkg	booléens/offsets	BSL-1.0	vcpkg (encapsulée)	github.com/AngusJohnson/Clipper2
nlohmann/json	via vcpkg	JSON projet	MIT	vcpkg (encapsulée)	github.com/nlohmann/json
minizip-ng	via vcpkg	ZIP projet	zlib	vcpkg (encapsulée)	github.com/zlib-ng/minizip-ng
Catch2 v3	via vcpkg	tests	BSL-1.0	vcpkg (dev)	github.com/catchorg/Catch2
Qt 6.8 LTS (Widgets/Gui/Core)	binaires officiels	interface	LGPL-3.0	liaison dynamique	qt.io

Les versions exactes sont verrouillées par la **baseline vcpkg** (`vcpkg.json`).

Qt et la LGPL-3.0

Qt est utilisé en **liaison dynamique** avec les DLL officielles non modifiées : elles sont remplaçables par l'utilisateur, aucune source Qt n'est modifiée, et seuls des modules LGPL sont utilisés (pas de module GPL-only ni commercial). Ce mode est compatible avec la distribution d'un logiciel Apache-2.0.

Licences de la chaîne documentaire

La documentation est générée avec `python-markdown` (BSD), `xhtml2pdf` (Apache-2.0) et `reportlab` (BSD), `svglib` (LGPL, utilisée comme outil externe, non liée au produit). Ces outils ne sont pas distribués avec le logiciel.

Implémentation associée

- `LICENSE, THIRD_PARTY_LICENSES.md`.
- `docs/phase0/03-bibliotheques-licences.md` (analyse d'origine, ADR-002).
- `docs/stitch-engine-research.md` (point juridique Ink/Stitch).

Annexe — Audit du dépôt

Cet audit est la **base factuelle** de toute la documentation : chaque affirmation technique du présent document est vérifiable dans le code réel du dépôt. Il a été établi par inspection directe de l'arborescence, des fichiers CMake, des en-têtes, des sources, des tests et des configurations.

Identité du projet

Élément	Valeur	Source
Nom	OpenStitch Studio (temporaire, remplaçable)	libs/core/include/openstitch/core/app_... (kAppName)
Version	0.1.0	même fichier (kAppVersion), CMakeLists.txt (project(... VERSION 0.1.0))
Licence	Apache-2.0	LICENSE, docs/phase0/03-bibliotheques-licences.r... (ADR-002)
Langage principal	C++23	CMakeLists.txt (CMAKE_CXX_STANDARD 23)
Plateforme	Windows 10/11 (cœur portable Linux)	CMakePresets.json (presets msvc, linux-core)
Interface	Qt 6 Widgets (LGPL, liaison dynamique)	apps/desktop/CMakeLists.txt
Gestion des dépendances	vcpkg (manifeste)	vcpkg.json, CMakePresets.json

Exécutables produits

Cible	Type	Fichier	Dépend de Qt
openstitch	Application de bureau	apps/desktop/	Oui
openstitch-cli	Outil en ligne de commande	apps/cli/main.cpp	Non

Sous-commandes CLI réellement définies (app.add_subcommand, apps/cli/main.cpp) : info, stats, dst2svg, stitchdebug.

Bibliothèques internes (14)

Chaque dossier de `libs/` est une cible CMake `openstitch::<nom>` :

Lib	Rôle	Dépendances internes clés	Encapsule
<code>core</code>	unités fortes, ids, <code>Result<T></code> , logging	—	fmt, spdlog
<code>geometry</code>	chemins, simplification, booléens/offsets, longueur d'arc	core	Clipper2
<code>image</code>	chargement + prétraitement non destructif	core	OpenCV
<code>segmentation</code>	quantification CIELAB, régions connexes	core, image	OpenCV
<code>vectorization</code>	régions → contours vectoriels	core, geometry, segmentation	OpenCV
<code>document</code>	modèle métier (projet, objets)	core, geometry, image, segmentation	—
<code>stitch</code>	commandes machine, statistiques	core	—
<code>stitch_generation</code>	running / tatami / satin	core, geometry, stitch, document	—
<code>stitch_analysis</code>	moteur de règles de validation	core, stitch	—
<code>optimization</code>	ordre de couture (coût, stratégies)	core	—
<code>autodigitize</code>	image → objets éditables	vectorization, stitch_generation	—
<code>commands</code>	undo/redo (Command pattern)	document	—
<code>formats</code>	codec DST maison, SVG de diagnostic	core, stitch	fmt
<code>project_io</code>	format projet .osp	core, document, image	nlohmann/json, minizip-ng

Dépendances externes

Depuis `vcpkg.json` et `apps/desktop/CMakeLists.txt` : `fmt`, `spdlog`, `cli11`, `clipper2`, `nlohmann-json`, `minizip-ng`, `opencv4` (features core, png, jpeg, tiff), `catch2` (tests) ; **Qt 6.8** hors `vcpkg` (binaires officiels, variable `QT_ROOT`). Voir le chapitre *Licences*.

Principaux espaces de noms et types

- Espace racine `openstitch` ; sous-espaces `geometry`, `image`, `segmentation`, `vectorization`, `document`, `stitch`, `stitch_generation`, `stitch_analysis`, `optimization`, `autodigitize`, `commands`, `formats`, `project_io`, `desktop`.
- Unités fortes : `Micrometers`, `Millimeters`, `Pixels`, `Angle`, `Vec2um` (`libs/core/include/openstitch/core/units.hpp`).
- Identifiants : `ObjectId`, `RegionId`, `ColorId`, `ThreadId`, `IdGenerator<T>` (`libs/core/include/openstitch/core/ids.hpp`).
- Erreurs : `Result<T> = std::expected<T, Error>` (`.../core/error.hpp`).
- Document : `Project`, `VectorObject`, `EmbroideryObject`, `StitchParams` (variant `RunningStitchParams` | `TatamiParams` | `SatinParams`), `Canvas` (`libs/document/include/openstitch/document/`).
- Points : `StitchCommand`, `CommandType`, `StitchSequence`, `StitchStats` (`libs/stitch/include/openstitch/stitch/sequence.hpp`).

Formats pris en charge

Format	Import	Export	Module
PNG, JPEG, BMP, TIFF	Oui	— (export projet uniquement)	<code>image</code> (OpenCV)
DST (Tajima)	Oui	Oui	<code>formats/src/dst.cpp</code>
SVG (diagnostic)	—	Oui	<code>formats/src/svg.cpp</code>
<code>.osp</code> (projet)	Oui	Oui	<code>project_io</code>

Interface graphique réellement implémentée

Menus construits dans `apps/desktop/main_window.cpp` (`buildMenus`, `buildAnalysisPanel`, `buildSimulationToolbar`, `buildOrderPanel`) : Fichier, Édition, Image, Segmentation, Broderie, Affichage, Analyse, plus un panneau d'ordre de couture (dock), un panneau d'analyse (dock) et une barre de simulation. Le détail est donné dans le *Guide utilisateur détaillé*.

Types d'objets de broderie

RunningStitchParams (point droit/double/triple), TatamiParams (remplissage), SatinParams (colonne satin), portés par le variant StitchParams
(libs/document/include/openstitch/document/embroidery_object.hpp).

Algorithmes présents (vérifiés dans le code)

Conversion RGB↔CIELAB et k-means (segmentation), composantes connexes 4-conn (segmentation), extraction de contours Suzuki-Abe (vectorization), simplification Douglas-Peucker (geometry/simplify.cpp), union/offset via Clipper2 (geometry/clean.cpp, geometry/offset.cpp), aplatissement Bézier adaptatif + longueur d'arc (geometry/polyline.cpp), running stitch par longueur d'arc avec découpe aux coins (stitch_generation/running_stitch.cpp), tatami scanline + routage par graphe (stitch_generation/tatami.cpp), satin à deux rails (stitch_generation/satin.cpp), règles d'analyse (stitch_analysis/analyze.cpp), ordre de couture (optimization/order.cpp), codec DST (formats/dst.cpp).

Tests présents

24 fichiers test_*.cpp sous tests/unit/<lib>/ et un test d'intégration tests/integration/test_pipeline.cpp. Total : **139 tests** au dernier passage (ctest). Framework : Catch2 v3. Voir le chapitre Tests.

Exemples et ressources

- SVG de diagnostic de référence : tests/golden/stitch-generation/ (formes de running stitch et remplissage anneau).
- Aucun jeu d'images d'exemple versionné à la racine examples/ à ce jour (*Information non déterminée dans le dépôt* : dossier examples/ non peuplé).

Fonctionnalités expérimentales / partielles / prévues

- **Prévu (architecture mentionnée, non implémenté)** : palette de fils réelle (thread_palette), profils de machine/cadres avancés, remplissages courbes / radiaux / spirale / motifs, sous-couches tatami, édition manuelle des points.
- **Partiel** : auto-satin (heuristique de largeur, pas d'axe médian) ; compensation (satin uniquement) ; routage tatami (parcours par graphe présent, underpath caché non implémenté — les liaisons hors-région sont des sauts).
- **Expérimental** : openstitch-cli stitchdebug (outil de diagnostic du moteur).

Marqueurs, incohérences

- Aucun `FIXME` critique repéré ; le `README.md` mentionne « 121 tests » alors que le compte réel est 139 (le `README` a été écrit à un jalon antérieur) — incohérence mineure, corrigée dans la présente documentation.
- Documents de conception : `docs/phase0/` (étude de cadrage, 14 ADR), `docs/stitch-engine-audit.md` et `docs/stitch-engine-research.md` (refonte du moteur de points).

Éléments non déterminés dans le dépôt

- URL de dépôt public : aucune (dépôt local uniquement).
- Fichier de logo : aucun logo bitmap/vectoriel dédié trouvé ; la couverture du PDF n'en utilise donc pas.
- Couverture de code chiffrée : non configurée.